Práctica 4- ENTRENAMIENTO, EVALUACIÓN Y DESPLIEGUE

TRABAJO REALIZADO POR:
JOSÉ A. RODRÍGUEZ LÓPEZ

Contenido

Entrenamiento y validación del modelo 6

1. Resumen 6

2. Introducción 6

3. Objetivos 7

3.1. Objetivo general 7

3.2. Objetivos específicos 7

4. Flujo de ejecución 8

5. Explicación del código 9

5.1. Estructura del notebook 9

5.2. Bloque de acceso a AWS S3 9

5.3. Módulo model.py: arquitectura y compilación 13

5.4. Modulo train_utils.py: tokenizador y dataset 16

5.5. Bloque de entrenamiento 19

5.6. Bloque de evaluación 25

5.7. Bloque de generación de texto 26

5.8. Descargar archivos generados 30

API y contenerización 32

1. Descripción general de la API 32

1.1. Flujo general de funcionamiento 32

2. Componentes principales 32

2.1. Framework de servicio 32

2.2. Modelo y tokenizador 33

2.3. Utilidades auxiliares 33

3. Seguridad implementada 33

3.1. Autenticación por API Key 33

3.2. Limitación de tasa 34

3.3. Encabezados de seguridad 34

4. Endpoints disponibles 35

4.1. Endpoint raíz 35

4.2. Endpoint de salud 35

4.3. Endpoint de generación 36

5. Lógica de generación de texto 37

5.1. _next_token_probs 37

5.2. _generate 37

6. Estrategias de decodificación 38

6.1. Greedy 38

6.2. Sampling 38

6.3. Top-k 38

7. Ejecución local de la API 39

8. Ejemplo de consumo de la API 39

8.1. Petición con curl 39

8.2. Petición con PowerShell 39

9. Contenerización con Docker 49
9.1. Objetivo de la contenerización 49
9.2. Estructura del Dockerfile 49
9.3. Construcción de la imagen 51
9.4. Ejecución del contenedor 51
9.5. Detención del contenedor 52
10. Dependencias necesarias 52
11. Ventajas y limitaciones de la solución 52
11.1. Ventajas 52
11.2. Limitaciones 53
12. Ejemplos de ejecución 53
12.1. Creación del contenedor 53
12.2. Ejecución del contenedor 54
12.3. Petición con curl 54
12.4. Petición con PowerShell 54
12.5. Petición interactiva 55
12.6. Detención del contenedor 56
Despliegue en la nube 57
1. Despliegue en la nube 57
1.1. Objetivo 57
1.2. Investigación y comparativa de opciones 57
1.3. Plataforma elegida y justificacion 59

1.4. Despliegue del contenedor (Azure Container Apps) 59

1.5. Verificacion funcional..... 66

1.6. Medidas de seguridad aplicadas 67

Entrenamiento y validación del modelo

1. Resumen

Este trabajo desarrolla un sistema de generación de texto basado en redes neuronales recurrentes tipo LSTM. Se ha sustituido el modelo Transformers, porque los resultados en la generación de texto son muy deficientes y para una generación medianamente aceptable, el conjunto de datos debe ser muy amplio, requiriendo una gran capacidad de cómputo y tiempo.

El flujo implementado cubre el ciclo completo de un proyecto de Machine Learning aplicado: preparación del entorno, obtención de datos desde AWS S3 privado, preprocesamiento, entrenamiento, evaluación cuantitativa y cualitativa, y descarga de los archivos generados.

El modelo se entrena sobre corpus textual tokenizado a nivel de palabra y se evalúa con perplejidad sobre un conjunto de prueba independiente. Como parte del análisis cualitativo, se comparan tres estrategias de decodificación: greedy search, sampling y top-k sampling.

2. Introducción

La tarea de modelado del lenguaje consiste en estimar la probabilidad condicional del siguiente token dado un contexto previo. En términos formales, el modelo aprende:

$$P(w_t \mid w_1, w_2, \dots, w_{t-1})$$

Para este proyecto se adopta una arquitectura LSTM multicapa, adecuada para secuencias y con menor complejidad operativa que enfoques más pesados en entornos de práctica.

3. Objetivos

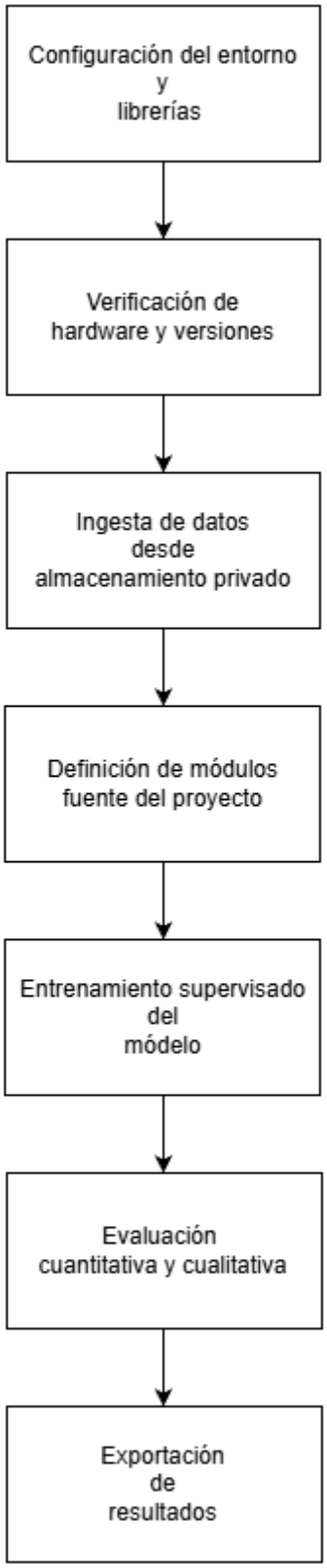
3.1. Objetivo general

Implementar un pipeline reproducible de generación de texto con entrenamiento, evaluación y descarga de archivos generados. Modulo

3.2. Objetivos específicos

1. Configurar un entorno ejecutable en Google Colab con soporte GPU.
2. Cargar un corpus privado desde AWS S3 mediante credenciales temporales de sesión.
3. Construir un tokenizador de palabras con vocabulario acotado.
4. Entrenar un modelo de lenguaje LSTM con callbacks de control.
5. Medir rendimiento con perplejidad en test.
6. Generar texto con distintos métodos de decodificación.
7. Empaquetar y exportar los archivos finales del proyecto.

4. Flujo de ejecución



5. Explicación del código

5.1. Estructura del notebook

El notebook PRACTICA_4.ipynb se compone de varias celdas markdown y código. Las celdas críticas son:

1. Descarga de corpus desde S3.
2. Generación de model.py.
3. Generación de train_utils.py.
4. Entrenamiento y evaluación.

5.2. Bloque de acceso a AWS S3

En la celda de S3 se implementa una carga segura y robusta:

1. Lectura de credenciales con getpass y variables de entorno.

```
import os # Librería para manejar variables de entorno
from getpass import getpass # Librería para ingresar contraseñas de forma segura

print("Configurar credenciales AWS para acceder a un bucket S3 privado")

# Solicitar al usuario que ingrese sus credenciales AWS de forma segura
aws_access_key_id = getpass("AWS_ACCESS_KEY_ID: ").strip()
aws_secret_access_key = getpass("AWS_SECRET_ACCESS_KEY: ").strip()
aws_default_region = input("AWS_DEFAULT_REGION [us-east-1]: ").strip() or "us-
east-1"

# Validar que se hayan ingresado las credenciales
if not aws_access_key_id or not aws_secret_access_key:
    raise ValueError("AWS_ACCESS_KEY_ID y AWS_SECRET_ACCESS_KEY son obligatorios
para este bucket privado.")

# Cargar las credenciales en variables de entorno para que boto3 pueda usarlas
```

```
os.environ["AWS_ACCESS_KEY_ID"] = aws_access_key_id
os.environ["AWS_SECRET_ACCESS_KEY"] = aws_secret_access_key
os.environ["AWS_DEFAULT_REGION"] = aws_default_region

print("Credenciales AWS cargadas en variables de entorno para esta sesión del
notebook")
```

```
Configurar credenciales AWS para acceder a un bucket S3 privado
AWS_ACCESS_KEY_ID: ··********
AWS_SECRET_ACCESS_KEY: ··********
AWS_DEFAULT_REGION [us-east-1]:
Credenciales AWS cargadas en variables de entorno para esta sesión del notebook
```

2. Definición de BUCKET_NAME, PREFIX y NOMBRE_ARCHIVO_CORPUS.

3. Construcción de clave final con prefijo: Practica_4/corpus.txt.

4. Descarga mediante boto3.client("s3").download_file(...).

5. Manejo de excepciones para: credenciales invalidas, objeto no encontrado (404/NoSuchKey) y errores generales de boto3.

```
import os # Librería para manejar rutas y directorios
import boto3 # Librería para interactuar con AWS S3
from botocore.exceptions import BotoCoreError, ClientError, NoCredentialsError,
PartialCredentialsError # Excepciones específicas de boto3
from pathlib import Path # Librería para manejar rutas de archivos

# Configuración del bucket S3 y rutas locales
BUCKET_NAME = "rodriguez-cloud-proyectos-ia"
PREFIX = "Practica_4/"
LOCAL_CORPUS_PATH = Path("data/corpus.txt")
REQUIRED_ENV_VARS = ["AWS_ACCESS_KEY_ID", "AWS_SECRET_ACCESS_KEY"]
NOMBRE_ARCHIVO_CORPUS = "corpus.txt"

# Construir la clave completa del objeto en S3 (incluyendo prefijo)
prefix_clean = PREFIX.strip("/")
S3_OBJECT_KEY = f"{prefix_clean}/{NOMBRE_ARCHIVO_CORPUS}" if prefix_clean else
NOMBRE_ARCHIVO_CORPUS
```

```
# Validar que las variables de entorno necesarias estén presentes
missing_vars = [name for name in REQUIRED_ENV_VARS if not os.getenv(name,
"").strip()]
if missing_vars:
    raise EnvironmentError(
        "Faltan credenciales AWS obligatorias. Ejecuta primero la celda anterior.
"
        f"Variables faltantes: {', '.join(missing_vars)}"
    )

# Crear directorios locales si no existen
LOCAL_CORPUS_PATH.parent.mkdir(parents=True, exist_ok=True)
Path("output").mkdir(parents=True, exist_ok=True)

# Inicializar el cliente S3 con las credenciales cargadas en variables de entorno
s3_client = boto3.client("s3", region_name=os.getenv("AWS_DEFAULT_REGION", "us-
east-1"))

# Descargar el corpus desde S3
try:
    # Descargar el archivo del corpus desde S3 a la ruta local especificada
    s3_client.download_file(BUCKET_NAME, S3_OBJECT_KEY, str(LOCAL_CORPUS_PATH))
except (NoCredentialsError, PartialCredentialsError):
    raise RuntimeError(
        "Las credenciales AWS no están completas o no son válidas. "
        "Vuelve a ejecutar la celda anterior con claves correctas."
    )
except ClientError as exc:
    error_code = exc.response.get("Error", {}).get("Code", "")
    if error_code in {"404", "NoSuchKey", "NotFound"}:
        # Mostrar pistas rápidas para encontrar la ruta correcta
        candidates = []
        try:
            response = s3_client.list_objects_v2(Bucket=BUCKET_NAME,
Prefix=prefix_clean + "/", MaxKeys=10)
            candidates = [obj["Key"] for obj in response.get("Contents", [])]
        except Exception:
```

```
            pass

        detalle = [
            f"No se encontró el objeto: s3://{BUCKET_NAME}/{S3_OBJECT_KEY}",
            "Verifica el nombre del archivo y el prefijo en S3.",
        ]
        if candidates:
            detalle.append("Ejemplos de claves disponibles bajo ese prefijo:")
            detalle.extend([f"- {key}" for key in candidates])

        raise RuntimeError("\n".join(detalle)) from exc

    raise RuntimeError(
        "No fue posible acceder al bucket privado en S3. "
        f"Detalle: {exc}"
    ) from exc
except BotoCoreError as exc:
    raise RuntimeError(
        "No fue posible acceder al bucket privado en S3. "
        f"Detalle: {exc}"
    ) from exc

# Mostrar información básica del corpus descargado
print(f"Corpus descargado desde s3://{BUCKET_NAME}/{S3_OBJECT_KEY}")
print(f"Guardado en: {LOCAL_CORPUS_PATH}")

size_mb = LOCAL_CORPUS_PATH.stat().st_size / (1024 * 1024)
print(f"Tamaño: {size_mb:.2f} MB")

# Leer y mostrar las primeras líneas del corpus para verificar su contenido
with LOCAL_CORPUS_PATH.open("r", encoding="utf-8", errors="ignore") as
file_handle:
    lines = []
    for _ in range(5):
        line = file_handle.readline()
        if not line:
            break
```

```
        lines.append(line)

print("\nPrimeras líneas del corpus:")
print("".join(lines))
```

```
Corpus descargado desde s3://rodriguez-cloud-proyectos-ia/Practica_4/corpus.txt
Guardado en: data/corpus.txt
Tamaño: 96.96 MB

Primeras líneas del corpus:
10 Herramientas de Inteligencia Artificial conversacionales
Ahora te dejaremos una pequeña lista de herramientas de inteligencia artificial conve
Ariana AI: bot de ChatGPT diseñado para ser utilizado en WhatsApp, Ariana está diseña
Bing Chat: Herramienta de inteligencia artificial conversacional desarrollada por Mic
CharacterAI: aplicación que te permite crear personajes de ficción con personalidades
```

5.3. Módulo model.py: arquitectura y compilación

Este archivo define dos elementos principales:

1. Clase LMConfig: encapsula hiperparámetros (vocab_size, embedding_dim, lstm_units, num_lstm_layers, dropout_rate, sequence_length) y permite serialización en JSON.

2. Funciones de construcción: build_language_model y compile_model.

Flujo de build_language_model:

1. Entrada de IDs de tokens con longitud fija sequence_length.

2. Capa Embedding con mask_zero=True para ignorar padding.

3. Bucle de capas LSTM con return_sequences=True para predecir en cada paso temporal.

4. LayerNormalization entre capas intermedias.

5. Dropout de salida para regularización.

6. Capa Dense con softmax sobre todo el vocabulario.

Compilación:

1. Optimizador Adam con learning_rate=5e-4 y clipnorm=1.0.

2. Función de perdida sparse_categorical_crossentropy.

3. Métrica de apoyo: accuracy.

```
# -*- coding: utf-8 -*-
"""
                  PRÁCTICA 4: Entrenamiento, Evaluación y Despliegue.
                          Script principal para la práctica 4
                        Programación de Inteligencia Artificial

Modelo de lenguaje para la práctica 4.

DESARROLLADO POR:   José A. Rodríguez López
FECHA: 27 de Marzo, 2026
PROYECTO: Programación de Inteligencia Artificial
==============================================================================
"""

import tensorflow as tf # Librería principal para Deep Learning
from tensorflow.keras import layers, Model # Librería para construir modelos con
Keras
import json # Librería para manejar archivos en formato JSON

class LMConfig:
    """Configuración del modelo de lenguaje."""

    def __init__(
        self,
        vocab_size=10000, # Tamaño del vocabulario (número de tokens únicos)
        embedding_dim=384, # Dimensión de los vectores de embedding para cada
token
        lstm_units=640, # Número de unidades en cada capa
        num_lstm_layers=3, # Número de capas en el modelo
        dropout_rate=0.25, # Tasa de dropout para regularización
        sequence_length=50, # Longitud de las secuencias de entrada para el modelo
    ):
```

```python
        self.vocab_size = vocab_size
        self.embedding_dim = embedding_dim
        self.lstm_units = lstm_units
        self.num_lstm_layers = num_lstm_layers
        self.dropout_rate = dropout_rate
        self.sequence_length = sequence_length

    # Métodos para guardar y cargar la configuración en formato JSON
    def save(self, path: str):
        with open(path, "w", encoding="utf-8") as f:
            json.dump(self.__dict__, f, indent=2)

    # Cargar la configuración desde un archivo JSON
    @classmethod
    def load(cls, path: str):
        with open(path, "r", encoding="utf-8") as f:
            data = json.load(f)
        return cls(**data)

# Funciones para construir y compilar el modelo de lenguaje
def build_language_model(config: LMConfig) -> Model:
    """Construye un modelo de lenguaje LSTM multi-capa."""
    inputs = layers.Input(shape=(config.sequence_length,), name="token_ids")

    # Capa de embedding para convertir IDs de tokens en vectores densos
    x = layers.Embedding(
        input_dim=config.vocab_size,
        output_dim=config.embedding_dim,
        mask_zero=True,
        name="embedding",
    )(inputs)

    # Mantener secuencia completa para predecir el siguiente token en cada paso.
    for i in range(config.num_lstm_layers):
        x = layers.LSTM(
            units=config.lstm_units,
            return_sequences=True,
```

```
            dropout=config.dropout_rate,
            recurrent_dropout=0.0,
            name=f"lstm_{i + 1}",
        )(x)
        if i < config.num_lstm_layers - 1:
            x = layers.LayerNormalization(name=f"layer_norm_{i + 1}")(x)

    # Capa de salida con softmax para predecir la probabilidad de cada token en el
vocabulario
    x = layers.Dropout(config.dropout_rate, name="output_dropout")(x)
    outputs = layers.Dense(config.vocab_size, activation="softmax",
name="output_probs")(x)

    # Crear el modelo Keras con las entradas y salidas definidas
    model = Model(inputs=inputs, outputs=outputs, name="LSTMLanguageModel")
    return model


def compile_model(model: Model, learning_rate: float = 5e-4) -> Model:
    """Compila el modelo."""
    optimizer = tf.keras.optimizers.Adam(learning_rate=learning_rate,
clipnorm=1.0)
    model.compile(optimizer=optimizer, loss="sparse_categorical_crossentropy",
metrics=["accuracy"])
    return model
```

5.4. Modulo train_utils.py: tokenizador y dataset

Incluye dos piezas fundamentales:

1. Clase TextTokenizer.
2. Función make_dataset.

Funcionamiento de TextTokenizer:

1. Construye vocabulario con frecuencia de palabras (Counter).
2. Reserva tokens especiales <PAD>, <UNK>, <BOS>, <EOS>.

3. Implementa codificación (encode) y decodificación (decode).

4. Permite guardar y cargar estado en JSON.

Funcionamiento de make_dataset:

1. Convierte lista de IDs en tf.data.Dataset.

2. Crea ventanas de tamaño seq_len + 1.

3. Separa entrada y objetivo con desplazamiento temporal: X = seq[:-1], Y = seq[1:].

4. Aplica shuffle (solo entrenamiento), batch y prefetch.

Esta transformación implementa el aprendizaje autoregresivo del siguiente token.

```
# -*- coding: utf-8 -*-
"""
                  PRÁCTICA 4: Entrenamiento, Evaluación y Despliegue.
                          Script principal para la práctica 4
                        Programación de Inteligencia Artificial

Modelo de lenguaje para la práctica 4.

DESARROLLADO POR:   José A. Rodríguez López
FECHA: 27 de Marzo, 2026
PROYECTO: Programación de Inteligencia Artificial
================================================================================
"""

import json # Librería para manejar archivos en formato JSON
from collections import Counter # Librería para contar la frecuencia de palabras
import tensorflow as tf # Librería principal para Deep Learning
import numpy as np # Librería para manejo de arrays y operaciones matemáticas

class TextTokenizer:
    """Tokenizador a nivel de palabra."""
    # Definir tokens especiales para el vocabulario
    PAD, UNK, BOS, EOS = "<PAD>", "<UNK>", "<BOS>", "<EOS>"
```

```python
    # Inicializar el tokenizador con un tamaño de vocabulario máximo
    def __init__(self, vocab_size: int = 10000):
        self.vocab_size = vocab_size
        self.word2idx = {}
        self.idx2word = {}

    def fit(self, text: str):
        """Construye el vocabulario."""
        words = text.lower().split()
        freq = Counter(words)
        specials = [self.PAD, self.UNK, self.BOS, self.EOS]
        vocab = specials + [w for w, _ in freq.most_common(self.vocab_size -
len(specials))]
        self.word2idx = {w: i for i, w in enumerate(vocab)}
        self.idx2word = {i: w for w, i in self.word2idx.items()}
        print(f"Vocabulario construido: {len(self.word2idx)} tokens")

    def encode(self, text: str):
        """Convierte texto a IDs de tokens, usando UNK para palabras fuera del
vocabulario."""
        unk = self.word2idx[self.UNK]
        return [self.word2idx.get(w, unk) for w in text.lower().split()]

    def decode(self, ids):
        """Convierte IDs de tokens a texto, usando UNK para tokens
desconocidos."""
        return " ".join(self.idx2word.get(i, self.UNK) for i in ids)

    def save(self, path: str):
        """Guarda el tokenizador en un archivo JSON."""
        with open(path, "w", encoding="utf-8") as f:
            json.dump({"word2idx": self.word2idx, "idx2word": {str(k): v for k, v
in self.idx2word.items()}}, f)

    @classmethod
    def load(cls, path: str):
        """Carga el tokenizador desde un archivo JSON."""
```

```
        with open(path, "r", encoding="utf-8") as f:
            data = json.load(f)
        tok = cls()
        tok.word2idx = data["word2idx"]
        tok.idx2word = {int(k): v for k, v in data["idx2word"].items()}
        tok.vocab_size = len(tok.word2idx)
        return tok

def make_dataset(token_ids, seq_len, batch_size, shuffle=True):
    """Crea un tf.data.Dataset para entrenamiento."""
    total = tf.data.Dataset.from_tensor_slices(token_ids)
    sequences = total.batch(seq_len + 1, drop_remainder=True)

    def split_xy(seq):
        """Dado un tensor de forma (seq_len + 1,), devuelve (seq_len,) para X y
(seq_len,) para Y."""
        return seq[:-1], seq[1:]

    # Aplicar la función de mapeo para dividir cada secuencia en entradas (X) y
etiquetas (Y)
    ds = sequences.map(split_xy, num_parallel_calls=tf.data.AUTOTUNE)
    # Mezclar los datos si se especifica, con un buffer grande para asegurar buena
aleatorización
    if shuffle:
        ds = ds.shuffle(buffer_size=10000)
    # Agrupar los datos en lotes de tamaño batch_size, descartando el último lote
si no tiene el tamaño completo
    ds = ds.batch(batch_size, drop_remainder=True)
    ds = ds.prefetch(tf.data.AUTOTUNE)
    return ds
```

5.5. Bloque de entrenamiento

En el notebook se realiza:

1. Carga del corpus y tokenización completa.

2. División 80/10/10 en train/val/test.

3. Construcción de datasets con seq_len=64 y batch_size=48.

4. Inicialización del modelo desde LMConfig.

5. Validación temprana de formas (yhat.shape debe ser 3D).

6. Entrenamiento con callbacks.

Callbacks implementados:

1. ModelCheckpoint por época en output/checkpoints/.

2. ModelCheckpoint para mejor modelo (output/best_model.keras).

3. ReduceLROnPlateau para ajuste automático de tasa de aprendizaje.

4. EarlyStopping con restore_best_weights=True.

Salida del entrenamiento:

1. output/model_final.keras.

2. Curva de loss y val_loss para inspeccion visual.

```
import importlib # Librería para recargar módulos.
import model as model_module # Importar el módulo del modelo para poder recargarlo
después de editarlo
from train_utils import TextTokenizer, make_dataset # Importar el tokenizador y la
función para crear datasets
import tensorflow as tf # Librería principal para Deep Learning
import time # Librería para medir el tiempo de entrenamiento
import glob # Librería para buscar archivos en el sistema
import os # Librería para manejar rutas y directorios
import matplotlib.pyplot as plt # Librería para visualización de datos

# Evita usar una versión cacheada del módulo model.py en el kernel.
model_module = importlib.reload(model_module)
LMConfig = model_module.LMConfig
build_language_model = model_module.build_language_model
compile_model = model_module.compile_model

# Buscar archivos de corpus en la carpeta data y cargar.
corpus_files = glob.glob("data/*")
if not corpus_files:
    print("No hay archivo de corpus. Por favor, sube uno en la celda anterior.")
```

```
else:
    corpus_path = corpus_files[0]
    print(f"Usando corpus: {corpus_path}")

    with open(corpus_path, "r", encoding="utf-8", errors="ignore") as f:
        text = f.read()
    print(f"Corpus: {len(text)} caracteres, ~{len(text.split())} palabras")

    tokenizer = TextTokenizer(vocab_size=15000) # Crear el tokenizador con un
vocabulario máximo de 15000 tokens
    tokenizer.fit(text) # Construir el vocabulario a partir del texto del corpus
    tokenizer.save("output/tokenizer.json") # Guardar el tokenizador

    token_ids = tokenizer.encode(text) # Convertir el texto del corpus a una lista
de IDs de tokens
    print(f"Tokens totales: {len(token_ids)}") # Mostrar el número total de tokens
en el corpus después de la tokenización

    # Dividir los datos en conjuntos de entrenamiento, validación y prueba (80%
train, 10% val, 10% test)
    train_end = int(0.8 * len(token_ids))
    val_end = int(0.9 * len(token_ids))
    train_ids = token_ids[:train_end]
    val_ids = token_ids[train_end:val_end]
    test_ids = token_ids[val_end:]
    print(f"Train: {len(train_ids)}, Val: {len(val_ids)}, Test: {len(test_ids)}")

    # Crear datasets de TensorFlow para entrenamiento, validación y prueba
    seq_len = 64
    batch_size = 48
    train_ds = make_dataset(train_ids, seq_len, batch_size, shuffle=True)
    val_ds = make_dataset(val_ids, seq_len, batch_size, shuffle=False)
    test_ds = make_dataset(test_ids, seq_len, batch_size, shuffle=False)

    # Construir y compilar el modelo de lenguaje
    config = LMConfig(
        vocab_size=len(tokenizer.word2idx),
        embedding_dim=384,
        lstm_units=640,
        num_lstm_layers=3,
        dropout_rate=0.25,
        sequence_length=seq_len,
    )
    config.save("output/model_config.json")

    # Construir el modelo de lenguaje utilizando la configuración definida y
compilarlo con una tasa de aprendizaje de 5e-4
```

```
    model = build_language_model(config)
    compile_model(model, learning_rate=5e-4)
    print("\nResumen del modelo:")
    model.summary()

    # Validación temprana de formas para evitar fallos en model.fit.
    x_preview, y_preview = next(iter(train_ds))
    yhat_preview = model(x_preview, training=False)
    print(f"Preview y shape: {y_preview.shape}, output shape:
{yhat_preview.shape}")
    if len(yhat_preview.shape) != 3:
        raise ValueError(
            f"Salida inválida del modelo: {yhat_preview.shape}. Se esperaba
(batch, seq_len, vocab_size)."
        )

    # Crear directorio para checkpoints si no existe
    os.makedirs("output/checkpoints", exist_ok=True)

    # Configurar callbacks para guardar checkpoints, reducir la tasa de
aprendizaje y detener el entrenamiento temprano
    print("\nIniciando entrenamiento...")
    t0 = time.time()

    checkpoint_every_epoch = tf.keras.callbacks.ModelCheckpoint(
        filepath="output/checkpoints/epoch_{epoch:02d}.keras",
        save_best_only=False,
        save_freq="epoch",
        verbose=1,
    )
    best_model_checkpoint = tf.keras.callbacks.ModelCheckpoint(
        filepath="output/best_model.keras",
        monitor="val_loss",
        save_best_only=True,
        verbose=1,
    )
    reduce_lr = tf.keras.callbacks.ReduceLROnPlateau(
        monitor="val_loss",
        factor=0.5,
        patience=2,
        min_lr=1e-5,
        verbose=1,
    )
    early_stop = tf.keras.callbacks.EarlyStopping(
        monitor="val_loss",
        patience=5,
        min_delta=1e-3,
```

```
        restore_best_weights=True,
        verbose=1,
    )

    # Entrenar el modelo utilizando los datasets de entrenamiento y validación,
con los callbacks configurados para guardar checkpoints, reducir la tasa de
aprendizaje y detener el entrenamiento temprano si no hay mejora en la validación.
    history = model.fit(
        train_ds,
        validation_data=val_ds,
        epochs=20,
        callbacks=[checkpoint_every_epoch, best_model_checkpoint, reduce_lr,
early_stop],
    )

    # Medir el tiempo de entrenamiento y mostrarlo en minutos
    elapsed = (time.time() - t0) / 60
    print(f"\nEntrenamiento completado en {elapsed:.1f} minutos")

    # Guardar el modelo final después del entrenamiento
    model.save("output/model_final.keras")
    print("Modelo guardado en output/model_final.keras")

    # Visualizar la pérdida de entrenamiento y validación a lo largo de las épocas
    plt.figure(figsize=(10, 4))
    plt.plot(history.history["loss"], label="train_loss")
    plt.plot(history.history["val_loss"], label="val_loss")
    plt.title("Monitorización de la pérdida durante el entrenamiento")
    plt.xlabel("Época")
    plt.ylabel("Loss")
    plt.legend()
    plt.grid(True)
    plt.show()

    # Mostrar la pérdida final de entrenamiento y validación después del último
epoch
    if history.history.get("loss"):
        print(f"Loss final entrenamiento: {history.history['loss'][-1]:.4f}")
    if history.history.get("val_loss"):
        print(f"Loss final validación: {history.history['val_loss'][-1]:.4f}")
```

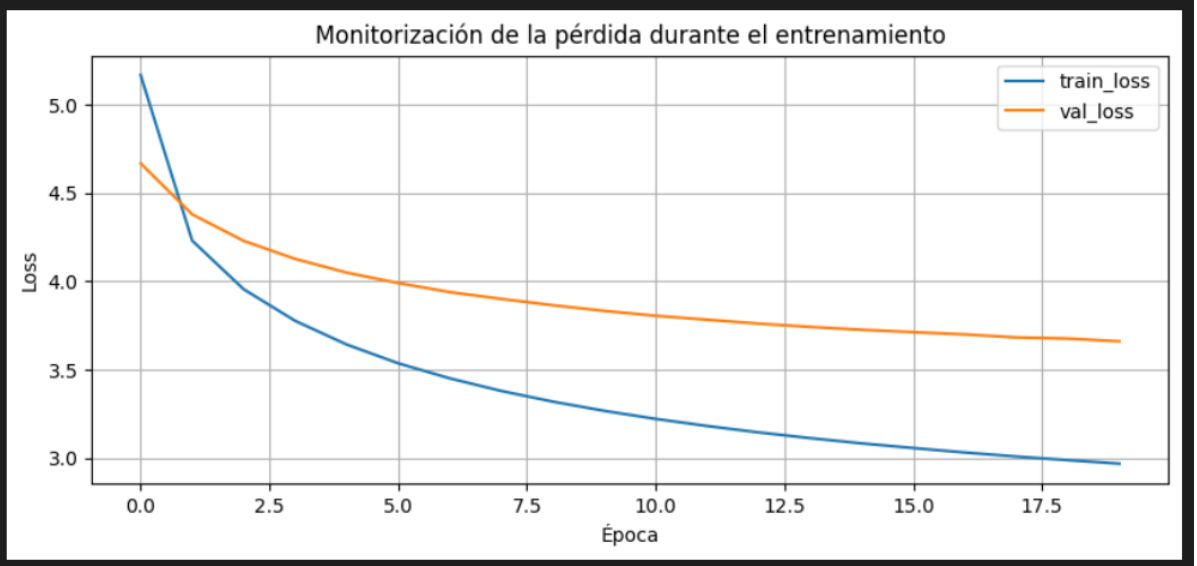
```
Usando corpus: data/corpus.txt
Corpus: 99381098 caracteres, ~15451938 palabras
Vocabulario construido: 15000 tokens
Tokens totales: 15451938
Train: 12361550, Val: 1545194, Test: 1545194

Resumen del modelo:


Model: "LSTMLanguageModel"
```

Layer (type)	Output Shape	Param #	Connected to
token_ids (InputLayer)	(None, 64)	0	-
embedding (Embedding)	(None, 64, 384)	5,760,000	token_ids[0][0]
not_equal_1 (NotEqual)	(None, 64)	0	token_ids[0][0]
lstm_1 (LSTM)	(None, 64, 640)	2,624,000	embedding[0][0], not_equal_1[0][0]
layer_norm_1 (LayerNormalizatio…	(None, 64, 640)	1,280	lstm_1[0][0]
lstm_2 (LSTM)	(None, 64, 640)	3,279,360	layer_norm_1[0][… not_equal_1[0][0]
layer_norm_2 (LayerNormalizatio…	(None, 64, 640)	1,280	lstm_2[0][0]
lstm_3 (LSTM)	(None, 64, 640)	3,279,360	layer_norm_2[0][… not_equal_1[0][0]
output_dropout (Dropout)	(None, 64, 640)	0	lstm_3[0][0]
output_probs (Dense)	(None, 64, 15000)	9,615,000	output_dropout[0…

```
 Total params: 24,560,280 (93.69 MB)


 Trainable params: 24,560,280 (93.69 MB)


 Non-trainable params: 0 (0.00 B)

Preview y shape: (48, 64), output shape: (48, 64, 15000)

Iniciando entrenamiento...
Epoch 1/20
3962/3962 ━━━━━━━━━━━━━━━━━━━━ 0s 218ms/step - accuracy: 0.1587 - loss: 5.8082
Epoch 1: saving model to output/checkpoints/epoch_01.keras

Epoch 1: finished saving model to output/checkpoints/epoch_01.keras

Epoch 1: val_loss improved from None to 4.66744, saving model to output/best_model.keras

Epoch 1: finished saving model to output/best_model.keras
3962/3962 ━━━━━━━━━━━━━━━━━━━━ 934s 234ms/step - accuracy: 0.1963 - loss: 5.1667 - val_accura
Epoch 2/20
3962/3962 ━━━━━━━━━━━━━━━━━━━━ 0s 218ms/step - accuracy: 0.2373 - loss: 4.3891
Epoch 2: saving model to output/checkpoints/epoch_02.keras
```

```
Restoring model weights from the end of the best epoch: 20.

Entrenamiento completado en 323.2 minutos
Modelo guardado en output/model_final.keras
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...




```
Loss final entrenamiento: 2.9685
Loss final validación: 3.6604
```

5.6. Bloque de evaluación

La función compute_perplexity calcula la loss media en test y posteriormente:

$$extPerplejidad = e^{\text{loss media}}$$

Interpretación aplicada en el notebook:

1. < 30: excelente.
2. < 100: buena.
3. < 300: aceptable.
4. >= 300: necesita mejoras de datos o entrenamiento.

```
import math # Librería para operaciones matemáticas
from train_utils import TextTokenizer # Importar el tokenizador para cargarlo y
usarlo en la evaluación
```

```python
# Cargar el modelo final guardado después del entrenamiento y el tokenizador para
evaluar el modelo en el conjunto de prueba
model = tf.keras.models.load_model("output/model_final.keras")
tokenizer = TextTokenizer.load("output/tokenizer.json")

def compute_perplexity(model, dataset):
    """Calcula la pérdida promedio y la perplexidad del modelo en el dataset
dado."""
    loss_fn = tf.keras.losses.SparseCategoricalCrossentropy(from_logits=False)
    total_loss = 0.0
    n_batches = 0

    for x_batch, y_batch in dataset:
        probs = model(x_batch, training=False)
        loss = loss_fn(y_batch, probs)
        total_loss += float(loss)
        n_batches += 1

    avg_loss = total_loss / max(n_batches, 1)
    return math.exp(avg_loss), avg_loss

# Evaluar el modelo en el conjunto de prueba y calcular la pérdida promedio y la
perplexidad
ppl, avg_loss = compute_perplexity(model, test_ds)
print(f"\nLOSS MEDIA EN TEST: {avg_loss:.4f}")
print(f"PERPLEJIDAD EN TEST: {ppl:.2f}")

# Interpretar la perplexidad obtenida en el conjunto de prueba utilizando umbrales
comunes para modelos de lenguaje
if ppl < 30:
    print("Interpretación: excelente")
elif ppl < 100:
    print("Interpretación: buena")
elif ppl < 300:
    print("Interpretación: aceptable")
else:
    print("Interpretación: el modelo necesita más entrenamiento o más datos")
```

```
LOSS MEDIA EN TEST: 3.7649
PERPLEJIDAD EN TEST: 43.16
Interpretación: buena
```

5.7. Bloque de generación de texto

Se implementan funciones auxiliares para manejar la inferencia token a token:

1. _next_token_probs: obtiene distribución y bloquea tokens especiales no deseados.

2. _resolve_seq_len: armoniza longitud efectiva de entrada.

3. _ensure_seed_tokens: garantiza semilla valida.

4. _build_model_input: aplica right-padding para compatibilidad con mascara.

Estrategias de generación:

1. generate_greedy: selección determinista del máximo.

2. generate_sampling: muestreo estocástico con temperatura.

3. generate_top_k: restringe la muestra al subconjunto de mayor probabilidad.

El objetivo de estas pruebas es comparar coherencia frente a diversidad textual.

```
import numpy as np # Librería para manejo de arrays y operaciones matemáticas

def _next_token_probs(model, x, tokenizer, valid_pos=None):
    """Devuelve distribución del siguiente token y bloquea tokens especiales."""
    pred = model.predict(x, verbose=0)

    # Si la salida es (1, seq_len, vocab_size), tomar la posición válida para
predecir el siguiente token.
    if pred.ndim == 3:
        if valid_pos is None:
            valid_pos = pred.shape[1] - 1
        valid_pos = max(0, min(int(valid_pos), pred.shape[1] - 1))
        probs = pred[0, valid_pos, :]
    else:
        probs = pred[0]

    # Evitar que aparezcan tokens especiales en la salida final.
    blocked = [
        tokenizer.word2idx.get(tokenizer.PAD),
        tokenizer.word2idx.get(tokenizer.UNK),
        tokenizer.word2idx.get(tokenizer.BOS),
    ]
    probs = probs.copy()
    for idx in blocked:
        if idx is not None and 0 <= idx < len(probs):
            probs[idx] = 0.0
```

```
    # Normalizar las probabilidades para que sumen 1, evitando divisiones por cero
si todas las probabilidades son cero después de bloquear.
    total = probs.sum()
    if total <= 0:
        probs = np.ones_like(probs, dtype=np.float64)
        for idx in blocked:
            if idx is not None and 0 <= idx < len(probs):
                probs[idx] = 0.0
        probs /= probs.sum()
    else:
        probs = probs / total

    return probs

def _resolve_seq_len(model, seq_len=None):
    """Usa la longitud de entrada real del modelo si no se pasa una
manualmente."""
    return int(seq_len) if seq_len is not None else int(model.input_shape[1])

def _ensure_seed_tokens(tokens, tokenizer):
    """Garantiza al menos un token para iniciar la generación."""
    if tokens:
        return tokens
    bos = tokenizer.word2idx.get(tokenizer.BOS)
    return [int(bos)] if bos is not None else [0]

def _build_model_input(tokens, seq_len):
    """Right padding para compatibilidad con máscaras en LSTM/CuDNN."""
    window = tokens[-seq_len:]
    valid_len = len(window)
    padded = window + [0] * (seq_len - valid_len)
    x = np.array([padded], dtype=np.int32)
    return x, valid_len - 1

def generate_greedy(model, tokenizer, seed, n_words=50, seq_len=None):
    """Greedy search: siempre el token más probable."""
    seq_len = _resolve_seq_len(model, seq_len)
    tokens = _ensure_seed_tokens(tokenizer.encode(seed), tokenizer)

    for _ in range(n_words):
        x, valid_pos = _build_model_input(tokens, seq_len)
        probs = _next_token_probs(model, x, tokenizer, valid_pos=valid_pos)
        next_token = int(np.argmax(probs))
        tokens.append(next_token)

    return tokenizer.decode(tokens)
```

```python
def generate_sampling(model, tokenizer, seed, n_words=50, seq_len=None,
temperature=1.0):
    """Sampling: más variado que greedy."""
    seq_len = _resolve_seq_len(model, seq_len)
    tokens = _ensure_seed_tokens(tokenizer.encode(seed), tokenizer)

    for _ in range(n_words):
        x, valid_pos = _build_model_input(tokens, seq_len)
        probs = _next_token_probs(model, x, tokenizer, valid_pos=valid_pos)
        log_probs = np.log(probs + 1e-10) / temperature
        log_probs -= log_probs.max()
        scaled_probs = np.exp(log_probs)
        scaled_probs /= scaled_probs.sum()
        next_token = int(np.random.choice(len(scaled_probs), p=scaled_probs))
        tokens.append(next_token)

    return tokenizer.decode(tokens)

def generate_top_k(model, tokenizer, seed, n_words=50, seq_len=None, k=40,
temperature=0.8):
    """Top-K sampling: limita a los K tokens más probables."""
    seq_len = _resolve_seq_len(model, seq_len)
    tokens = _ensure_seed_tokens(tokenizer.encode(seed), tokenizer)

    for _ in range(n_words):
        x, valid_pos = _build_model_input(tokens, seq_len)
        probs = _next_token_probs(model, x, tokenizer, valid_pos=valid_pos)
        top_k_idx = np.argsort(probs)[-k:]
        top_k_probs = probs[top_k_idx]
        log_probs = np.log(top_k_probs + 1e-10) / temperature
        log_probs -= log_probs.max()
        scaled = np.exp(log_probs)
        scaled /= scaled.sum()
        next_token = int(np.random.choice(top_k_idx, p=scaled))
        tokens.append(next_token)

    return tokenizer.decode(tokens)

# Generar con diferentes estrategias
seed = "la inteligencia artificial"
print(f"SEED: '{seed}'")
print("=" * 70)

print("\nESTRATEGIA 1: Greedy (determinista)")
text1 = generate_greedy(model, tokenizer, seed, n_words=40)
print(text1[:150] + "...")
```

```
print("\nESTRATEGIA 2: Sampling (T=0.8, creativo)")
text2 = generate_sampling(model, tokenizer, seed, n_words=40, temperature=0.8)
print(text2[:150] + "...")

print("\nESTRATEGIA 3: Top-K (k=40, controlado)")
text3 = generate_top_k(model, tokenizer, seed, n_words=40, k=40, temperature=0.8)
print(text3[:150] + "...")
```

```
SEED: 'la inteligencia artificial'
======================================================================

ESTRATEGIA 1: Greedy (determinista)
la inteligencia artificial (ia) ha sido necesariamente adoptada aprovechando áreas de desarro

ESTRATEGIA 2: Sampling (T=0.8, creativo)
la inteligencia artificial y la psicología cognitiva que han integrado la teoría de la inform

ESTRATEGIA 3: Top-K (k=40, controlado)
la inteligencia artificial y las ciencias de la computación, la computación ubicua o la compu
```

5.8. Descargar archivos generados

Descarga todos los ficheros generados (modelo, tokenizador, etc....) comprimidos en un ZIP a la carpeta **Descargas** del equipo local.

```
from google.colab import files # Librería para descargar archivos desde Colab al
PC
import os # Librería para manejar rutas y directorios
import zipfile # Librería para crear archivos ZIP

# Archivos a incluir en la descarga
ficheros_a_descargar = [
    "output/model_final.keras",
    "output/best_model.keras",
    "output/tokenizer.json",
    "output/model_config.json",
    "model.py",
    "train_utils.py",
]

# Crear un archivo ZIP con los archivos listados para facilitar la descarga
zip_path = "practica4_archivos.zip"

print("Creando ZIP con los archivos generados...")
with zipfile.ZipFile(zip_path, "w", zipfile.ZIP_DEFLATED) as zf:
    for f in ficheros_a_descargar:
        if os.path.exists(f):
            zf.write(f)
```

```
            print(f"  Añadido: {f}")
        else:
            print(f"  No encontrado (omitido): {f}")

size_mb = os.path.getsize(zip_path) / (1024 * 1024)
print(f"\nZIP creado: {zip_path} ({size_mb:.2f} MB)")
print("Descargando a la carpeta Descargas del equipo...")
files.download(zip_path)
print("Descarga finalizada")
```

```
Creando ZIP con los archivos generados...
  Añadido: output/model_final.keras
  Añadido: output/best_model.keras
  Añadido: output/tokenizer.json
  Añadido: output/model_config.json
  Añadido: model.py
  Añadido: train_utils.py

ZIP creado: practica4_archivos.zip (519.97 MB)
Descargando a la carpeta Descargas del equipo...

Descarga finalizada
```

API y contenerización

1. Descripción general de la API

La API está implementada en el archivo api_simple.py utilizando **FastAPI**. Su función principal es cargar un modelo de lenguaje previamente entrenado y ofrecer un endpoint HTTP que permita generar texto a partir de una semilla proporcionada por el cliente.

1.1. Flujo general de funcionamiento

Cuando la aplicación arranca, realiza las siguientes acciones:

1. Lee variables de entorno para configurar autenticación y limitación de tasa.
2. Carga el modelo entrenado desde output/model_final.keras.
3. Carga el tokenizador desde output/tokenizer.json.
4. Inicializa la aplicación FastAPI.
5. Registra middleware de seguridad y CORS.
6. Expone endpoints para estado y generación de texto.

En términos prácticos, la API actúa como una capa de inferencia sobre el modelo, de forma que el usuario no necesita interactuar directamente con TensorFlow ni con la lógica interna de tokenización.

2. Componentes principales

2.1. Framework de servicio

El framework utilizado es **FastAPI**, elegido por las siguientes ventajas:

- definición clara de endpoints.

- validación automática de entrada con Pydantic.
- documentación interactiva en /docs.
- integración sencilla con uvicorn para despliegue.
- buen rendimiento para servicios ligeros de inferencia.

2.2. Modelo y tokenizador

La API depende de dos artefactos externos:

- output/model_final.keras: modelo entrenado de generación de texto.
- output/tokenizer.json: tokenizador serializado.

Ambos deben existir antes de arrancar el servicio. Si alguno falta, la importación del módulo fallará al inicio, ya que la carga se realiza al arrancar la aplicación y no de forma diferida.

2.3. Utilidades auxiliares

El archivo train_utils.py aporta la clase TextTokenizer, utilizada para:

- convertir texto en secuencias numéricas.
- reconstruir texto a partir de tokens generados.
- acceder a tokens especiales como <PAD>, <UNK> y <BOS>.

3. Seguridad implementada

3.1. Autenticación por API Key

El acceso al endpoint de generación está protegido mediante el encabezado HTTP:

```
X-API-Key
```

La validación se realiza comparando el valor recibido con la variable de entorno API_KEY.

Si la clave no existe o no coincide, la API responde con:

- código 401 Unauthorized.
- mensaje: API Key inválida o ausente.

3.2. Limitación de tasa

Se utiliza slowapi para restringir el número de peticiones al endpoint de generación. El límite se configura con la variable de entorno:

```
RATE_LIMIT
```

Por ejemplo, si RATE_LIMIT=10, se permiten 10 solicitudes por minuto.

Si slowapi no está instalado, el código utiliza una implementación de respaldo para no romper la ejecución, aunque en ese caso la limitación real no queda aplicada.

3.3. Encabezados de seguridad

Se añade un middleware HTTP que incorpora estos encabezados en cada respuesta:

- X-Content-Type-Options: nosniff
- X-Frame-Options: DENY
- X-XSS-Protection: 1; mode=block
- Cache-Control: no-store

Estos encabezados ayudan a reducir riesgos básicos de exposición en navegador y a evitar almacenamiento en caché de respuestas.

4. Endpoints disponibles

4.1. Endpoint raíz

Ruta: GET /

Devuelve una respuesta sencilla con el nombre de la API y el acceso a la documentación automática.

Ejemplo de respuesta:

```
{
  "message": "API de Generación de Texto",
  "docs": "/docs"
}
```

4.2. Endpoint de salud

Ruta: GET /health

Permite verificar que el servicio está activo y que la configuración principal se ha cargado correctamente.

Ejemplo de respuesta:

```
{
  "status": "ok",
  "model_loaded": true,
  "tokenizer_loaded": true,
  "rate_limiter_enabled": true
}
```

Este endpoint es útil para:

- validación rápida tras el arranque.
- comprobación de despliegue en Docker.
- pruebas automáticas de disponibilidad.

4.3. Endpoint de generación

Ruta: POST /generate

Este es el endpoint principal del sistema. Requiere autenticación por X-API-Key.

Cuerpo de entrada

```
{
  "seed": "el conocimiento",
  "n_words": 30,
  "strategy": "top_k",
  "temperature": 0.9,
  "top_k": 40
}
```

Parámetros admitidos

- seed: texto inicial desde el que comienza la generación.
- n_words: número de palabras a generar. Rango permitido: de 1 a 200.
- strategy: estrategia de decodificación. Valores válidos:
    - greedy
    - sampling
    - top_k
- temperature: controla la aleatoriedad del muestreo. Rango: de 0.1 a 2.0.
- top_k: tamaño del subconjunto de tokens candidatos cuando la estrategia es top_k.

Respuesta esperada

```
{
  "seed": "el conocimiento",
  "generated_text": "el conocimiento ...",
  "strategy": "top_k",
  "n_words_generated": 30,
```

```
  "elapsed_ms": 152.37
}
```

Validación de entrada

La API utiliza modelos Pydantic para validar automáticamente:

- longitud mínima y máxima del texto de entrada.
- rangos numéricos permitidos.
- estrategias válidas de generación.

Esto reduce errores de uso y evita que lleguen al modelo valores mal formados.

5. Lógica de generación de texto

La generación se apoya en dos funciones internas:

5.1. _next_token_probs

Esta función:

1. ejecuta la predicción del modelo.
2. obtiene la distribución de probabilidad del siguiente token.
3. bloquea tokens especiales no deseados.
4. renormaliza las probabilidades restantes.

Bloquear tokens como padding o desconocidos mejora la limpieza de la salida generada.

5.2. _generate

Esta función realiza la generación autoregresiva palabra por palabra:

1. tokeniza la semilla.

2. ajusta la longitud de entrada al tamaño esperado por el modelo.
3. predice el siguiente token.
4. elige el token según la estrategia seleccionada.
5. repite el proceso hasta alcanzar n_words.
6. decodifica el resultado final a texto.

6. Estrategias de decodificación

La API implementa tres estrategias para seleccionar el siguiente token:

6.1. Greedy

Selecciona siempre el token con mayor probabilidad.

Ventaja: salida estable y determinista.

Desventaja: puede generar texto menos diverso y más repetitivo.

6.2. Sampling

Realiza muestreo sobre toda la distribución de probabilidad, ajustada por temperatura.

Ventaja: mayor variedad textual.

Desventaja: puede introducir incoherencias si la temperatura es alta.

6.3. Top-k

Restringe la selección a los k tokens más probables y luego aplica muestreo.

Ventaja: equilibrio entre coherencia y diversidad.

Desventaja: requiere ajustar adecuadamente top_k y temperature.

7. Ejecución local de la API

Una vez instaladas las dependencias del archivo requirements.txt, puede arrancarse el servicio localmente con:

```
uvicorn api_simple:app --host 0.0.0.0 --port 8000
```

Si se desea configurar variables de entorno antes de iniciar:

En PowerShell

```
$env:API_KEY="tu_clave_segura"
$env:RATE_LIMIT="10"
uvicorn api_simple:app --host 0.0.0.0 --port 8000
```

Después del arranque, la documentación interactiva queda accesible en:

- http://localhost:8000/docs

8. Ejemplo de consumo de la API

8.1. Petición con curl

```
curl -X POST "http://localhost:8000/generate" \
  -H "Content-Type: application/json" \
  -H "X-API-Key: tu_clave_segura" \
  -d '{
    "seed": "la inteligencia artificial",
    "n_words": 20,
    "strategy": "sampling",
    "temperature": 0.8,
    "top_k": 40
  }'
```

8.2. Petición con PowerShell

```
$headers = @{
  "Content-Type" = "application/json"
  "X-API-Key" = "tu_clave_segura"
```

```
}

$body = @{
  seed = "la inteligencia artificial"
  n_words = 20
  strategy = "sampling"
  temperature = 0.8
  top_k = 40
} | ConvertTo-Json

Invoke-RestMethod -Method Post -Uri "http://localhost:8000/generate" -
Headers $headers -Body $body
```

```
# -*- coding: utf-8 -*-
"""
                 PRÁCTICA 4: Entrenamiento, Evaluación y Despliegue.
                       API de Generación de Texto con FastAPI
                       Programación de Inteligencia Artificial

Este módulo implementa una API RESTful para generar texto utilizando un modelo de
lenguaje entrenado. La API ofrece un endpoint para generar texto a partir de una
semilla dada, con opciones de configuración para la estrategia de generación y el
número de palabras a generar. Se incluyen medidas de seguridad como autenticación
por API Key y limitación de tasa para proteger el servicio.

DESARROLLADO POR:   José A. Rodríguez López
FECHA: 27 de Marzo, 2026
PROYECTO: Programación de Inteligencia Artificial
================================================================================
"""

from fastapi import FastAPI, HTTPException, Request, Security # Librerías de
FastAPI para crear la API, manejar excepciones, solicitudes y seguridad
from fastapi.middleware.cors import CORSMiddleware # Librería para manejar CORS
(Cross-Origin Resource Sharing)
from fastapi.security.api_key import APIKeyHeader # Librería para manejar
autenticación por API Key
from fastapi.responses import JSONResponse # Librería para enviar respuestas JSON
from pydantic import BaseModel, Field, field_validator # Librerías de Pydantic
para definir modelos de datos y validación de campos
```

```python
import numpy as np # Librería para manejar arrays y operaciones numéricas,
utilizada para procesar las probabilidades de generación de texto
import tensorflow as tf # Librería para cargar el modelo de lenguaje entrenado y
realizar predicciones
from train_utils import TextTokenizer # Importación de la clase TextTokenizer
desde el módulo train_utils, utilizada para tokenizar y detokenizar texto
import os # Librería para manejar variables de entorno, utilizada para configurar
la API Key y el límite de tasa desde el entorno de ejecución
import time # Librería para medir el tiempo de generación de texto, utilizada para
calcular el tiempo transcurrido en la generación de texto y devolverlo en la
respuesta

# slowapi es opcional en entornos donde no esté instalado.
try:
    from slowapi import Limiter, _rate_limit_exceeded_handler # Librerías de
slowapi para implementar limitación de tasa (rate limiting) en la API, protegiendo
el servicio contra abusos y sobrecargas
    from slowapi.errors import RateLimitExceeded # Excepción que se lanza cuando
se excede el límite de tasa
    from slowapi.util import get_remote_address # Función para obtener la
dirección IP del cliente, utilizada para aplicar la limitación de tasa basada en
la IP
    SLOWAPI_AVAILABLE = True
except ModuleNotFoundError:
    SLOWAPI_AVAILABLE = False

    class RateLimitExceeded(Exception):
        """
        Excepción ficticia para manejar casos donde slowapi no está disponible
        Args:
            Exception: La clase base de excepciones en Python, utilizada para
definir una excepción personalizada que se lanzará cuando se exceda el límite de
tasa, aunque en este caso no se aplicará realmente debido a la ausencia de
slowapi.
        """
        pass

    def get_remote_address(request):
        """
        Función ficticia para obtener la dirección IP del cliente cuando slowapi
no está disponible.
        Args:
            request (Request): La solicitud HTTP entrante.
        Returns:
            str: Una cadena fija "local" para indicar que la limitación de tasa no
se está aplicando realmente.
        """
```

```
        return "local"

    def _rate_limit_exceeded_handler(request, exc):
        """
        Manejador ficticio para cuando se excede el límite de tasa y slowapi no
está disponible.
        Args:
            request (Request): La solicitud HTTP que causó la excepción.
            exc (Exception): La excepción que se lanzó.
        """
        return JSONResponse(status_code=429, content={"detail": "Rate limit
exceeded"})

    class Limiter:
        """
        Clase ficticia para manejar la limitación de tasa cuando slowapi no está
disponible.
        """
        def __init__(self, key_func=None):
            """
            Inicializa el limitador de tasa con una función de clave
personalizada, aunque en este caso no hará nada debido a la ausencia de slowapi.
            Args:
                key_func (callable, optional): Una función que toma una solicitud
y devuelve una clave única para aplicar la limitación de tasa, como la dirección
IP del cliente. En este caso, se puede proporcionar una función personalizada,
pero no se aplicará realmente debido a la ausencia de slowapi
            """
            self.key_func = key_func

        def limit(self, _rule):
            """
            Decorador ficticio para aplicar la limitación de tasa a un endpoint,
aunque en este caso no hará nada debido a la ausencia de slowapi.
            Args:
                _rule (str): La regla de limitación de tasa, por ejemplo
"10/minute".
            """
            def decorator(func):
                return func

            return decorator


API_KEY = os.getenv("API_KEY", "%KhJh-yj44k[RMuJpy") # La clave de API se obtiene
de una variable de entorno, con un valor predeterminado para desarrollo. En
```

```
producción, se recomienda establecer esta variable de entorno a un valor seguro y
único para proteger el acceso a la API.
RATE_LIMIT = os.getenv("RATE_LIMIT", "10") # El límite de tasa. En producción, se
recomienda ajustar este valor según las necesidades del servicio y la capacidad
del servidor para manejar solicitudes concurrentes.
MODEL_PATH = "output/model_final.keras" # La ruta al modelo de lenguaje
TOKENIZER_PATH = "output/tokenizer.json" # La ruta al tokenizador

# Cargar el modelo y el tokenizador al iniciar la aplicación.
model = tf.keras.models.load_model(MODEL_PATH)
tokenizer = TextTokenizer.load(TOKENIZER_PATH)
seq_len = model.input_shape[1]

# Configuración de la aplicación FastAPI, incluyendo limitación de tasa, manejo de
excepciones y middleware para seguridad y CORS.
limiter = Limiter(key_func=get_remote_address)
app = FastAPI(title="API de Generación de Texto", version="1.0.0")
app.state.limiter = limiter
app.add_exception_handler(RateLimitExceeded, _rate_limit_exceeded_handler)
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Middleware para agregar encabezados de seguridad a todas las respuestas,
protegiendo contra ataques comunes como XSS, clickjacking y sniffing de contenido,
además de deshabilitar el almacenamiento en caché para garantizar que los clientes
siempre reciban la respuesta más reciente.
@app.middleware("http")
async def add_security_headers(request: Request, call_next):
    response = await call_next(request)
    response.headers["X-Content-Type-Options"] = "nosniff"
    response.headers["X-Frame-Options"] = "DENY"
    response.headers["X-XSS-Protection"] = "1; mode=block"
    response.headers["Cache-Control"] = "no-store"
    return response

# Configuración de seguridad para la autenticación por API Key. Se define un
encabezado personalizado "X-API-Key" que los clientes deben incluir en sus
solicitudes para autenticarse. La función verify_api_key verifica que la clave
proporcionada sea válida y coincide con la clave configurada en el servidor,
devolviendo un error 401 si la clave es inválida o ausente.
api_key_header = APIKeyHeader(name="X-API-Key", auto_error=False)
```

```python
def verify_api_key(key: str = Security(api_key_header)) -> str:
    """
    Verifica la clave de API proporcionada en el encabezado de la solicitud. Si la
clave es válida, se devuelve; de lo contrario, se lanza una excepción HTTP 401.
    Args:
        key (str, optional): La clave de API proporcionada en el encabezado de la
solicitud.
    Raises:
        HTTPException: Se lanza una excepción HTTP 401 si la clave de API es
inválida o ausente.
    Returns:
        str: La clave de API válida.
    """
    if not key or key != API_KEY:
        raise HTTPException(status_code=401, detail="API Key inválida o ausente")
    return key

class GenerateRequest(BaseModel):
    """
    Modelo de datos para la solicitud de generación de texto. Define los campos
necesarios para configurar la generación de texto, incluyendo la semilla inicial,
el número de palabras a generar, la estrategia de generación, la temperatura para
muestreo y el valor de top_k para limitar las opciones de generación. Se incluyen
validaciones para asegurar que los valores proporcionados sean válidos y dentro de
los rangos permitidos.
    Args:
        seed (str): La semilla inicial para la generación de texto. Debe ser una
cadena de texto con una longitud mínima de 1 y máxima de 500 caracteres.
        n_words (int): El número de palabras a generar. Debe ser un entero entre 1
y 200, con un valor predeterminado de 50.
        strategy (str): La estrategia de generación a utilizar. Puede ser "greedy"
para selección del token con mayor probabilidad, "sampling" para muestreo basado
en la distribución de probabilidades, o "top_k" para limitar las opciones a los k
tokens más probables. El valor predeterminado es "greedy".
        temperature (float): El valor de temperatura para el muestreo, que
controla la aleatoriedad de la generación. Debe ser un número flotante entre 0.1 y
2.0, con un valor predeterminado de 1.0.
        top_k (int): El valor de top_k para limitar las opciones de generación a
los k tokens más probables. Debe ser un entero entre 1 y 500, con un valor
predeterminado de 40. Este campo solo se utiliza si la estrategia seleccionada es
"top_k".
    """
    seed: str = Field(..., min_length=1, max_length=500)
    n_words: int = Field(default=50, ge=1, le=200)
    strategy: str = Field(default="greedy")
    temperature: float = Field(default=1.0, ge=0.1, le=2.0)
    top_k: int = Field(default=40, ge=1, le=500)
```

```
    @field_validator("strategy")
    @classmethod
    def validate_strategy(cls, value):
        """
        Valida que la estrategia de generación proporcionada sea una de las
opciones permitidas: "greedy", "sampling" o "top_k". Si la estrategia no es
válida, se lanza una excepción ValueError.
        Args:
            value (str): La estrategia de generación a validar.
        Raises:
            ValueError: Se lanza una excepción si la estrategia no es una de las
opciones permitidas.
        Returns:
            str: La estrategia de generación válida.
        """
        allowed = {"greedy", "sampling", "top_k"}
        if value not in allowed:
            raise ValueError(f"La estrategia debe ser una de {allowed}")
        return value

class GenerateResponse(BaseModel):
    """
    Generación de texto. Define los campos que se incluirán en la respuesta de la
API después de generar el texto, incluyendo la semilla utilizada, el texto
generado, la estrategia de generación, el número de palabras generadas y el tiempo
transcurrido en milisegundos para realizar la generación. Este modelo se utiliza
para estructurar la respuesta JSON que se enviará al cliente después de procesar
la solicitud de generación de texto.
    Args:
        BaseModel (_type_): La clase base de Pydantic para definir modelos de
datos, utilizada para crear un modelo de respuesta estructurado que se convertirá
automáticamente en JSON al ser devuelto por la API.
    """
    seed: str
    generated_text: str
    strategy: str
    n_words_generated: int
    elapsed_ms: float

def _next_token_probs(x: np.ndarray) -> np.ndarray:
    """
    Calcula las probabilidades del siguiente token dado un input de tokens. Esta
función toma una matriz de tokens como entrada, realiza una predicción utilizando
el modelo de lenguaje cargado y procesa las probabilidades para asegurarse de que
los tokens bloqueados (como PAD, UNK y BOS) tengan una probabilidad de 0. Si todas
las probabilidades resultan en 0 después de bloquear los tokens, se asigna una
```

```
distribución uniforme a los tokens restantes. Finalmente, se devuelve un array de
probabilidades normalizado para el siguiente token.
    Args:
        x (np.ndarray): Una matriz de tokens de entrada con forma (1, seq_len),
donde seq_len es la longitud de la secuencia que el modelo espera como entrada.
    Returns:
        np.ndarray: Un array de probabilidades normalizado para el siguiente
token, con forma (vocab_size,), donde vocab_size es el tamaño del vocabulario del
tokenizador. Las probabilidades de los tokens bloqueados (PAD, UNK, BOS) se
establecen en 0, y las probabilidades restantes se normalizan para que sumen 1.
    """
    pred = model.predict(x, verbose=0)
    probs = pred[0, -1, :] if pred.ndim == 3 else pred[0]

    blocked = [
        tokenizer.word2idx.get(tokenizer.PAD),
        tokenizer.word2idx.get(tokenizer.UNK),
        tokenizer.word2idx.get(tokenizer.BOS),
    ]
    probs = probs.copy()
    for idx in blocked:
        if idx is not None and 0 <= idx < len(probs):
            probs[idx] = 0.0

    total = probs.sum()
    if total <= 0:
        probs = np.ones_like(probs, dtype=np.float64)
        for idx in blocked:
            if idx is not None and 0 <= idx < len(probs):
                probs[idx] = 0.0
        probs /= probs.sum()
    else:
        probs = probs / total

    return probs

def _generate(req: GenerateRequest) -> str:
    """
    Genera texto a partir de una semilla dada utilizando la estrategia de
generación especificada en la solicitud. Esta función toma un objeto
GenerateRequest como entrada, que contiene la semilla inicial, la estrategia de
generación, el número de palabras a generar, la temperatura para muestreo y el
valor de top_k. La función procesa la semilla para obtener los tokens iniciales,
luego itera para generar el número especificado de palabras, utilizando la función
_next_token_probs para obtener las probabilidades del siguiente token y aplicando
la estrategia de generación seleccionada (greedy, sampling o top_k) para elegir el
```

```
siguiente token. Finalmente, se decodifica la secuencia de tokens generados y se
devuelve como texto.
    Args:
        req (GenerateRequest): La solicitud de generación de texto que contiene la
semilla, la estrategia de generación, el número de palabras a generar, la
temperatura y el valor de top_k.
    Returns:
        str: El texto generado a partir de la semilla proporcionada y la
estrategia de generación especificada.
    """
    tokens = tokenizer.encode(req.seed)

    for _ in range(req.n_words):
        padded = tokens[-seq_len:]
        padded = [0] * (seq_len - len(padded)) + padded
        x = np.array([padded])
        probs = _next_token_probs(x)

        if req.strategy == "greedy":
            next_token = int(np.argmax(probs))
        elif req.strategy == "sampling":
            log_probs = np.log(probs + 1e-10) / req.temperature
            log_probs -= log_probs.max()
            scaled = np.exp(log_probs)
            scaled /= scaled.sum()
            next_token = int(np.random.choice(len(scaled), p=scaled))
        else:
            k = min(req.top_k, len(probs))
            top_k_idx = np.argsort(probs)[-k:]
            top_k_probs = probs[top_k_idx]
            log_probs = np.log(top_k_probs + 1e-10) / req.temperature
            log_probs -= log_probs.max()
            scaled = np.exp(log_probs)
            scaled /= scaled.sum()
            next_token = int(np.random.choice(top_k_idx, p=scaled))

        tokens.append(next_token)

    return tokenizer.decode(tokens)

@app.get("/")
async def root():
    """
    Endpoint raíz de la API. Devuelve un mensaje de bienvenida y una referencia a
la documentación de la API.
    Returns:
```

```
        dict: Un diccionario con un mensaje de bienvenida y la ruta a la
documentación de la API.
    """
    return {"message": "API de Generación de Texto", "docs": "/docs"}

@app.get("/health")
async def health():
    """
    Endpoint de salud de la API. Devuelve información sobre el estado del modelo,
el tokenizador y la configuración de limitación de tasa. Este endpoint se puede
utilizar para verificar que la API está funcionando correctamente y que los
componentes necesarios (modelo y tokenizador) están cargados y listos para generar
texto.
    Returns:
        dict: Un diccionario con el estado de la API, incluyendo el estado del
modelo, el tokenizador y la limitación de tasa.
    """
    return {
        "status": "ok",
        "model_loaded": True,
        "tokenizer_loaded": True,
        "rate_limiter_enabled": SLOWAPI_AVAILABLE,
    }

@app.post("/generate", response_model=GenerateResponse,
dependencies=[Security(verify_api_key)])
@limiter.limit(f"{RATE_LIMIT}/minute")
async def generate(request: Request, body: GenerateRequest):
    """
    Endpoint para generar texto a partir de una semilla dada utilizando la
estrategia de generación especificada en la solicitud.
    Args:
        request (Request): La solicitud HTTP entrante.
        body (GenerateRequest): La solicitud de generación de texto que contiene
la semilla, la estrategia de generación, el número de palabras a generar, la
temperatura y el valor de top_k.
    Returns:
        GenerateResponse: La respuesta de generación de texto que contiene la
semilla, el texto generado, la estrategia utilizada, el número de palabras
generadas y el tiempo transcurrido en milisegundos.
    """
    start = time.perf_counter()
    generated = _generate(body)
    elapsed_ms = (time.perf_counter() - start) * 1000
    return GenerateResponse(
        seed=body.seed,
        generated_text=generated,
```

```
        strategy=body.strategy,
        n_words_generated=body.n_words,
        elapsed_ms=round(elapsed_ms, 2),
    )
```

9. Contenerización con Docker

9.1. Objetivo de la contenerización

La contenerización permite empaquetar la API junto con su entorno de ejecución para que el servicio pueda desplegarse de manera más reproducible en distintos equipos.

Las ventajas principales son:

- aislamiento de dependencias.
- facilidad de despliegue.
- reproducibilidad del entorno.
- menor fricción entre desarrollo y entrega.

9.2. Estructura del Dockerfile

El archivo Dockerfile define el proceso de construcción de la imagen:

1. parte de la imagen base python:3.13-slim.
2. define /app como directorio de trabajo.
3. copia requirements.txt.
4. instala dependencias con pip.
5. copia api_simple.py y train_utils.py.
6. crea el directorio output.
7. copia modelo y tokenizador a las rutas esperadas,

8. expone el puerto 8000.

9. arranca la API con uvicorn.

```
#                 PRÁCTICA 4: Entrenamiento, Evaluación y Despliegue.
#          Contenedor Docker para la API de Generación de Texto con FastAPI
#                       Programación de Inteligencia Artificial
#
# Dockerfile para construir una imagen que contenga la API de generación de
# texto desarrollada en esta práctica. La imagen se basa en Python 3.13 y
# expone el puerto 8000 para servir la API a través de uvicorn. Se incluyen las
# dependencias necesarias, el código fuente de la API, y los artefactos del
# modelo entrenado (modelo y tokenizador) para asegurar que la API funcione
# correctamente al desplegarla en un contenedor Docker.
#
# DESARROLLADO POR:   José A. Rodríguez López
# FECHA: 27 de Marzo, 2026
# PROYECTO: Programación de Inteligencia Artificial
#==============================================================================

# Creación del contenedor
# docker build -t practica4-api .

# Ejecución del contenedor.
# docker run -p 8000:8000 -e API_KEY=%KhJh-yj44k[RMuJpy -e RATE_LIMIT=10
practica4-api

# Detención del contenedor.
# docker stop practica4-api

# Imagen base con Python 3.13 en variante ligera.
FROM python:3.13-slim

# Directorio de trabajo dentro del contenedor.
WORKDIR /app

# Actualizar pip a la última versión para evitar problemas de instalación.
RUN pip install --upgrade pip
```

```
# Copia e instala dependencias para aprovechar cache de capas.
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copia el codigo de la API y utilidades necesarias.
COPY api_simple.py train_utils.py ./

# Crea la carpeta de artefactos y copia modelo/tokenizador en la ruta que usa la
API.
RUN mkdir -p output
COPY modelo/model_final.keras ./output/model_final.keras
COPY tokenizer/tokenizer.json ./output/tokenizer.json

# Puerto donde uvicorn expone la API.
EXPOSE 8000

# Variables de entorno: API_KEY debe pasarse al ejecutar por seguridad.
# RATE_LIMIT es configurable en tiempo de ejecución.
ENV RATE_LIMIT=10

# Comando de inicio del servicio FastAPI.
CMD ["uvicorn", "api_simple:app", "--host", "0.0.0.0", "--port", "8000"]
```

9.3. Construcción de la imagen

El propio Dockerfile propone este comando:

```
docker build -t practica4-api .
```

9.4. Ejecución del contenedor

Ejemplo de ejecución:

```
docker run -p 8000:8000 -e API_KEY=tu_clave_segura -e RATE_LIMIT=10
practica4-api
```

Con ello, la API quedaría accesible desde el host en:

- http://localhost:8000

9.5. Detención del contenedor

```
docker stop practica4-api
```

10. Dependencias necesarias

El archivo requirements.txt incluye:

- tensorflow
- numpy
- fastapi
- uvicorn
- pydantic
- slowapi
- python-dotenv
- httpx

Estas dependencias cubren el motor de inferencia, la API web, la validación de datos y herramientas de prueba o soporte.

11. Ventajas y limitaciones de la solución

11.1. Ventajas

- despliegue sencillo del modelo como servicio REST.
- validación automática de entradas.
- documentación interactiva de FastAPI.
- seguridad básica mediante API Key y rate limiting.

- posibilidad de ejecutar localmente o dentro de Docker.

11.2. Limitaciones

- la carga del modelo ocurre al importar el módulo, por lo que cualquier ausencia de artefactos impide iniciar la API.
- la clave API tiene un valor por defecto en código, apropiado solo para desarrollo.
- la política CORS permite cualquier origen (*), lo que conviene restringir en entornos más controlados.
- el tamaño de la imagen puede ser elevado debido a TensorFlow.

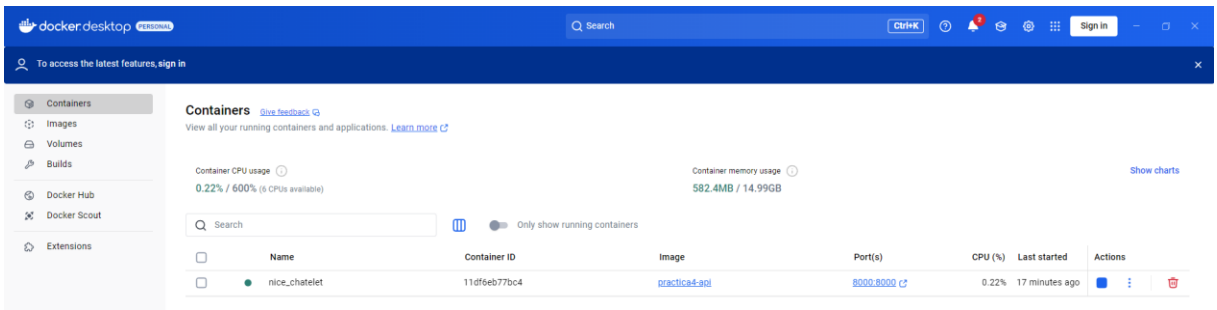
12. Ejemplos de ejecución

12.1. Creación del contenedor

```
PS C:\PRACTICA_4> docker build -t practica4-api .
[+] Building 151.8s (14/14) FINISHED                                                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                    0.0s
 => => transferring dockerfile: 2.15kB                                                                                  0.0s
 => [internal] load metadata for docker.io/library/python:3.13-slim                                                     0.3s
 => [internal] load .dockerignore                                                                                       0.0s
 => => transferring context: 2B                                                                                         0.0s
 => [1/9] FROM docker.io/library/python:3.13-slim@sha256:739e7213785e88c0f702dcdc12c0973afcbd606dbf021a589cab77d6  0.1s
 => => resolve docker.io/library/python:3.13-slim@sha256:739e7213785e88c0f702dcdc12c0973afcbd606dbf021a589cab77d6  0.1s
 => [internal] load build context                                                                                       0.0s
 => => transferring context: 18.08kB                                                                                    0.0s
 => CACHED [2/9] WORKDIR /app                                                                                           0.0s
 => [3/9] RUN pip install --upgrade pip                                                                                 4.0s
 => [4/9] COPY requirements.txt .                                                                                       0.1s
 => [5/9] RUN pip install --no-cache-dir -r requirements.txt                                                           55.3s
 => [6/9] COPY api_simple.py train_utils.py ./                                                                          0.3s
 => [7/9] RUN mkdir -p output                                                                                           0.4s
 => [8/9] COPY modelo/model_final.keras ./output/model_final.keras                                                      1.4s
 => [9/9] COPY tokenizer/tokenizer.json ./output/tokenizer.json                                                         0.1s
 => exporting to image                                                                                                 89.6s
 => => exporting layers                                                                                                70.5s
 => => exporting manifest sha256:4ed8a702cff490113f21e0d3d5d6a65b1a18477ec6f1caa47c57707c404cf201                       0.0s
 => => exporting config sha256:778591d5ea83f32511db27f118cb2f6b3c0600c749d6a054412371409d9e5ca6                         0.0s
 => => exporting attestation manifest sha256:e7a9d65fc845a3361e87e3b297132938fa4190e5160b6397c0402a4e84068211           0.1s
 => => exporting manifest list sha256:b3f3ec869441a2c4aeaab5cbd497a00e1888a5ca202fa226b00a2d038b5d0ca3                  0.0s
 => => naming to docker.io/library/practica4-api:latest                                                                 0.0s
 => => unpacking to docker.io/library/practica4-api:latest                                                             18.9s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/jkmdt1v34tznztan3vkwilgs7
```

12.2. Ejecución del contenedor

```
PS C:\PRACTICA_4> docker run -p 8000:8000 -e API_KEY=%KhJh-yj44k[RMuJpy -e RATE_LIMIT=10 practica4-api
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1774708063.355739       1 cudart_stub.cc:31] Could not find cuda drivers on your machine, GPU will not be us
ed.
I0000 00:00:1774708063.418224       1 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use available CPU
 instructions in performance-critical operations.
To enable the following instructions: AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler fl
ags.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1774708065.025642       1 cudart_stub.cc:31] Could not find cuda drivers on your machine, GPU will not be us
ed.
E0000 00:00:1774708065.260381       1 cuda_platform.cc:52] failed call to cuInit: INTERNAL: CUDA error: Failed call to c
uInit: UNKNOWN ERROR (303)
INFO:     Started server process [1]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```



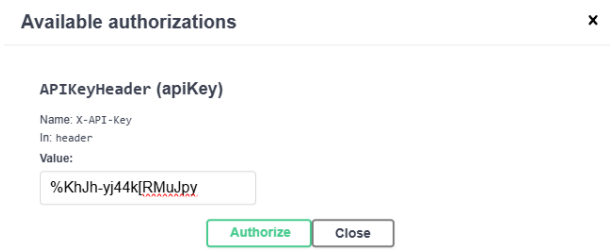
12.3. Petición con curl

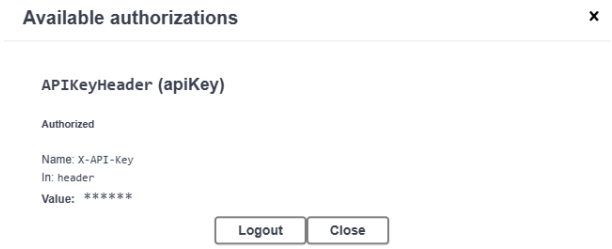
```
PS C:\Users\j0626> curl -X POST http://localhost:8000/generate `
>>   -H "X-API-Key: %KhJh-yj44k[RMuJpy" `
>>   -H "Content-Type: application/json" `
>>   -d '{"seed": "La inteligencia artificial", "n_words": 50, "strategy": "sampling", "temperature": 0.8}'
{"seed":"La inteligencia artificial","generated_text":"la inteligencia artificial es un campo que se encarga de estudiar
 la estructura del sistema y al menos una acción humana que pueda ser usada para traducir entre las representaciones en
otras ontologías o bases de datos. comenzó como una mezcla de elementos básicos de las ontologías opencyc y está constru
ido de acuerdo","strategy":"sampling","n_words_generated":50,"elapsed_ms":12134.91}
```

12.4. Petición con PowerShell

```
PS C:\Users\j0626> $headers = @{
>>   "Content-Type" = "application/json"
>>   "X-API-Key" = "%KhJh-yj44k[RMuJpy"
>> }
PS C:\Users\j0626> $body = @{
>>   seed = "la inteligencia artificial"
>>   n_words = 20
>>   strategy = "sampling"
>>   temperature = 0.8
>>   top_k = 40
>> } | ConvertTo-Json
PS C:\Users\j0626> Invoke-RestMethod -Method Post -Uri "http://localhost:8000/generate" -Headers $headers -Body $body

seed             : la inteligencia artificial
generated_text   : la inteligencia artificial también se ha añadido una nueva área a la empresa denominada inteligencia artificial y que se encargan de analizar cómo
strategy         : sampling
n_words_generated : 20
elapsed_ms       : 5195,62
```

12.5. Petición interactiva





POST /generate Generate

Endpoint para generar texto a partir de una semilla dada utilizando la estrategia de generación especificada en la solicitud. Args: request (Request): La solicitud HTTP entrante. body (GenerateRequest): La solicitud de generación de texto que contiene la semilla, la estrategia de generación, el número de palabras a generar, la temperatura y el valor de top_k. Returns: GenerateResponse: La respuesta de generación de texto que contiene la semilla, el texto generado, la estrategia utilizada, el número de palabras generadas y el tiempo transcurrido en milisegundos.

Parameters

Cancel Reset

No parameters

Request body required

application/json

Edit Value | Schema

```
{
  "seed": "Las redes neuronales",
  "n_words": 50,
  "strategy": "greedy",
  "temperature": 1,
  "top_k": 40
}
```

Execute Clear

Curl

```
curl -X 'POST' \
  'http://localhost:8000/generate' \
  -H 'accept: application/json' \
  -H 'X-API-Key: %KhJh-yj44k[RMuJpy' \
  -H 'Content-Type: application/json' \
  -d '{
  "seed": "Las redes neuronales",
  "n_words": 50,
  "strategy": "greedy",
  "temperature": 1,
  "top_k": 40
}'
```

Request URL

```
http://localhost:8000/generate
```

Server response

Code	Details
200	Response body

```
{
  "seed": "Las redes neuronales",
  "generated_text": "las redes neuronales artificiales se han utilizado para encontrar un conjunto de variables ocultas que no se han definido sobre un conjunto particular de funciones que corre
sponden a las variables independientes de las variables padres y su relación con las variables padres y las variables de salida, y el tiempo de propagación de",
  "strategy": "greedy",
  "n_words_generated": 50,
  "elapsed_ms": 12057.32
}
```

Download

12.6. Detención del contenedor

```
PS C:\Users\j0626> docker stop nice_chatelet
nice_chatelet
```

Despliegue en la nube

1. Despliegue en la nube

1.1. Objetivo

Este apartado documenta la estrategia de despliegue en la nube de la API de generación de texto entrenada en la práctica. El objetivo final es publicar una URL accesible de forma externa para consumir el endpoint de generación de texto.

1.2. Investigación y comparativa de opciones

1.2.1. IaaS (máquina virtual)

Una alternativa clásica es desplegar la API dentro de una máquina virtual en Azure, AWS o GCP, instalando Docker y ejecutando la imagen manualmente.

Ventajas:

- Control total del sistema operativo y red.

- Permite personalizaciones avanzadas.

- Fácil de entender para escenarios académicos iniciales.

Desventajas:

- Mayor carga operativa (actualizaciones, parches, firewall, monitoreo).

- Escalado menos automático.

- Mayor riesgo de mala configuración de seguridad si no se administra correctamente.

1.2.2. PaaS (Heroku)

Heroku permite desplegar aplicaciones web de forma sencilla con poca gestión de infraestructura.

Ventajas:

- Despliegue rápido y flujo simple.

- Menor esfuerzo DevOps inicial.

- Adecuado para demos y prototipos.

Desventajas:

- Menor control sobre la infraestructura.

- Costes y límites de plataforma a medio plazo.

- Menos orientado a cargas de inferencia de IA en crecimiento.

1.2.3. Servicios específicos para contenedores/IA

Se consideran tres opciones:

1.2.3.1. Azure Container Apps

- Despliegue directo de imagen Docker.

- Escalado automático por demanda.

- Gestión de secretos integrada.

1.2.3.2. AWS App Runner

- Exposición rápida de servicios web en contenedor.

- Autoescalado administrado.

1.2.3.3. SageMaker Endpoints

- Servicio orientado específicamente a modelos de ML en AWS.

- Muy potente para MLOps, pero con mayor complejidad para una API FastAPI sencilla.

1.3. Plataforma elegida y justificacion

Se elige **Azure Container Apps** para esta práctica.

Motivos:

- Se adapta de forma natural al flujo actual basado en Docker.

- Permite publicar URL publica con configuración simple.

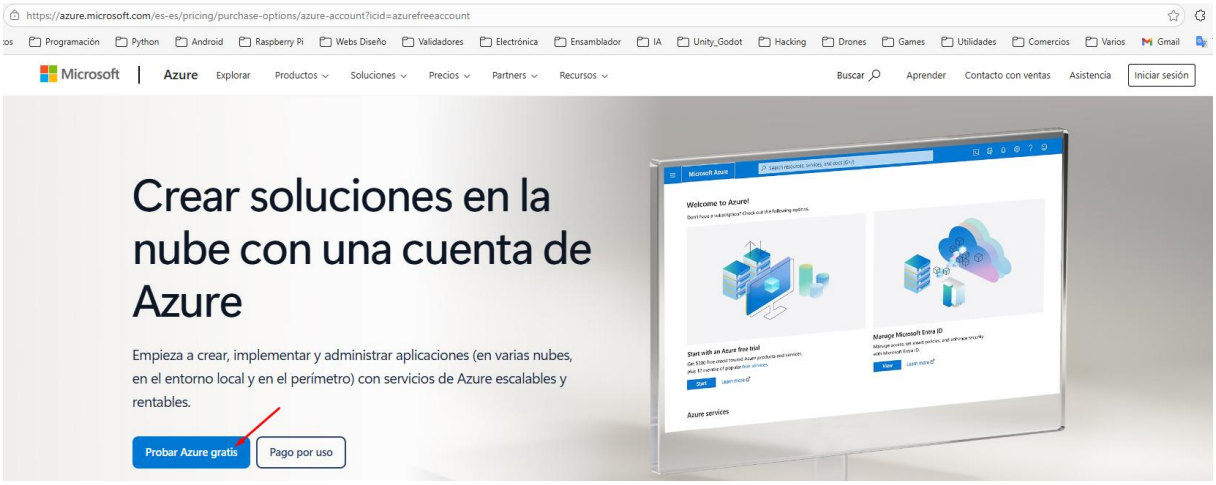
- Ofrece escalado automático sin gestionar máquinas virtuales.

- Integra secretos para no exponer claves sensibles en la imagen.

1.4. Despliegue del contenedor (Azure Container Apps)

1.4.1. Requisitos previos

- Cuenta activa de Azure.



Microsoft Azure [redacted] Cerrar sesión

Microsoft Azure

Creación de una cuenta de Azure gratuita

Una cuenta gratuita de Azure le proporciona acceso para explorar capacidades y desarrollar pruebas de concepto. Tu cuenta también incluye:

- USD200 de crédito para usar en los primeros 30 días
- Cantidades mensuales gratuitas de más de 20 servicios populares durante 12 meses
- Cantidades mensuales gratuitas de más de 65 servicios siempre gratuitos
- Protección de gasto: no se realizará ningún cargo automáticamente en su tarjeta de crédito hasta que cambie a la modalidad de pago por uso

Paso 1 de 3

Información de perfil

¿Cómo planea utilizar su cuenta de Azure?

- (●) Para uso personal
- ( ) Para su uso en relación con una organización, universidad, grupo de investigación u ONG

País o región

[redacted]

Elija la ubicación que coincida con su dirección de facturación. **No podrá cambiar la selección posteriormente.** Si su país no aparece en la lista, la oferta no está disponible en su región. Más información

Nombre

[redacted]

Apellido

[redacted]

Dirección de correo electrónico para notificaciones importantes

[redacted]

Teléfono

[redacted]

Microsoft Azure [redacted] Cerrar sesión

Microsoft Azure

Creación de una cuenta de Azure gratuita

Una cuenta gratuita de Azure le proporciona acceso para explorar capacidades y desarrollar pruebas de concepto. Tu cuenta también incluye:

- USD200 de crédito para usar en los primeros 30 días
- Cantidades mensuales gratuitas de más de 20 servicios populares durante 12 meses
- Cantidades mensuales gratuitas de más de 65 servicios siempre gratuitos
- Protección de gasto: no se realizará ningún cargo automáticamente en su tarjeta de crédito hasta que cambie a la modalidad de pago por uso

Paso 2 de 3

Información de dirección

Dirección (línea 1)

[redacted]

Dirección línea 2 (Opcional)

Ciudad

[redacted]

Provincia

[redacted]

Código postal

[redacted]

Entiendo que Microsoft puede ponerse en contacto conmigo sobre mi cuenta gratuita.

- [x] Represento que soy la persona identificada en esta cuenta. Acepto los términos y condiciones de la Contrato de cliente de Microsoft incluidos los Términos de usuario individuales complementarios.
- [ ] Me gustaría recibir información, sugerencias y ofertas sobre Azure y otros productos y servicios de Microsoft.
- [ ] Quiero que Microsoft comparta mi información con determinados partners para poder recibir información pertinente sobre sus productos y servicios.

Microsoft Azure [redacted] Cerrar sesión

Microsoft Azure

Creación de una cuenta de Azure gratuita

Una cuenta gratuita de Azure le proporciona acceso para explorar capacidades y desarrollar pruebas de concepto. Tu cuenta también incluye:

- USD200 de crédito para usar en los primeros 30 días
- Cantidades mensuales gratuitas de más de 20 servicios populares durante 12 meses
- Cantidades mensuales gratuitas de más de 65 servicios siempre gratuitos
- Protección de gasto: no se realizará ningún cargo automáticamente en su tarjeta de crédito hasta que cambie a la modalidad de pago por uso

Paso 3 de 3

Información de pago

No se le cobrará a menos que cambie a precios de pago por uso.

Se aceptan las tarjetas siguientes:

VISA

Nombre del titular de la tarjeta

[redacted]

Número de tarjeta

[redacted] VISA

Expira

[redacted]

CVV

[redacted] ¿Qué es un CVV?

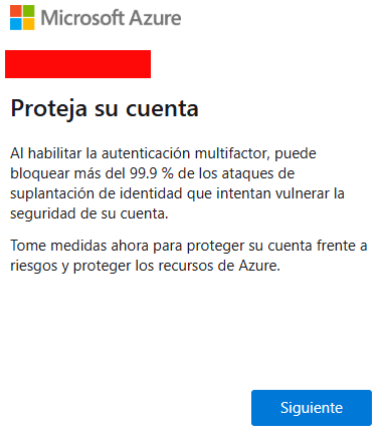
Dirección (línea 1)

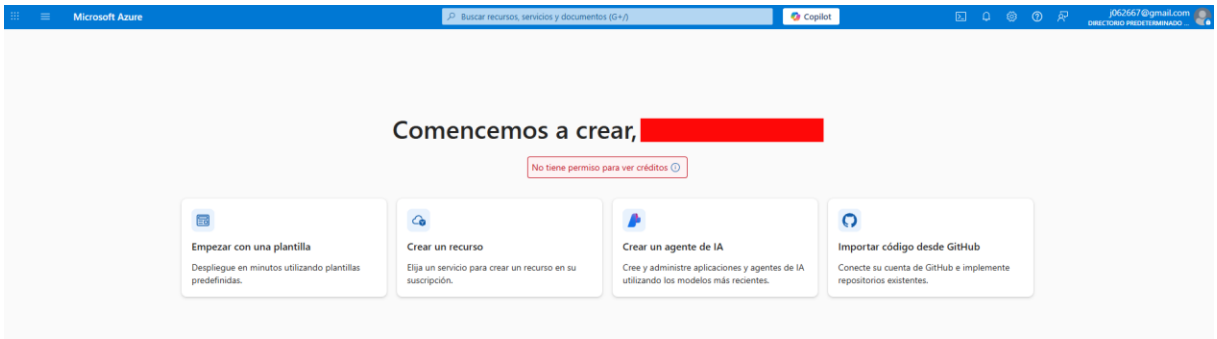
[redacted]

Dirección línea 2 (Opcional)

Ciudad

[redacted]





- Azure CLI instalado.

```
PS C:\Users\j0626> winget install --id Microsoft.AzureCLI
Encontrado Microsoft Azure CLI [Microsoft.AzureCLI] Versión 2.84.0
El propietario de esta aplicación le concede una licencia.
Microsoft no es responsable, ni tampoco concede ninguna licencia de paquetes de terceros.
Descargando https://azcliprod.blob.core.windows.net/msi/azure-cli-2.84.0-x64.msi
  ██████████████████████████████  64.8 MB / 64.8 MB
El hash del instalador se verificó correctamente
Iniciando instalación de paquete...
Instalado correctamente
Notas: Winget installs the 64-bit CLI on 64-bit OS by default now. If you have used the 32-
bit CLI before, please follow this guide to migrate to 64-bit version: https://learn.micros
oft.com/cli/azure/install-azure-cli-windows#migrate-to-64-bit-azure-cli
```

```
PS C:\Users\j0626> az login
Select the account you want to log in with. For more information on login with Azure CLI, s
ee https://go.microsoft.com/fwlink/?linkid=2271136

Retrieving tenants and subscriptions for the selection...
No subscriptions found for j062667@gmail.com.
PS C:\Users\j0626> az login
Select the account you want to log in with. For more information on login with Azure CLI, s
ee https://go.microsoft.com/fwlink/?linkid=2271136

Retrieving tenants and subscriptions for the selection...

[Tenant and subscription selection]

No     Subscription name     Subscription ID                       Tenant
-----  --------------------  ------------------------------------  ------------------------
-
[1] *  Azure subscription 1  652a3cac-df0f-4b90-a24d-d281ce918841  Directorio predeterminad
o

The default is marked with an *; the default tenant is 'Directorio predeterminado' and subs
cription is 'Azure subscription 1' (652a3cac-df0f-4b90-a24d-d281ce918841).

Select a subscription and tenant (Type a number or Enter for no changes):
```

```
PS C:\Users\j0626> az provider show -n Microsoft.ContainerRegistry --query registrationStat
e
"Registering"
```

```
PS C:\Users\j0626> az provider show -n Microsoft.ContainerRegistry --query registrationStat
e
"Registered"
```

- Docker Desktop instalado y autenticado.

- Imagen local construida correctamente.

1.4.2. Construcción de imagen local

En la raíz del proyecto:

```
docker build -t practica4-api .
```

1.4.3. Creación de recursos en Azure

Ejemplo con región West Europe:

```
az group create -n rg-practica4 -l westeurope
az acr create -n acrpractica4 -g rg-practica4 --sku Basic
az acr login -n acrpractica4
```

```
PS C:\Users\j0626> az group create -n rg-practica4 -l westeurope
{
  "id": "/subscriptions/652a3cac-df0f-4b90-a24d-d281ce918841/resourceGroups/rg-practica4",
  "location": "westeurope",
  "managedBy": null,
  "name": "rg-practica4",
  "properties": {
    "provisioningState": "Succeeded"
  },
  "tags": null,
  "type": "Microsoft.Resources/resourceGroups"
}
```

```
PS C:\Users\j0626> az acr create -n acrpractica4 -g rg-practica4 --sku Basic
{
  "adminUserEnabled": false,
  "anonymousPullEnabled": false,
  "autoGeneratedDomainNameLabelScope": "Unsecure",
  "creationDate": "2026-03-28T16:44:22.157497+00:00",
  "dataEndpointEnabled": false,
  "dataEndpointHostNames": [],
  "encryption": {
    "keyVaultProperties": null,
    "status": "disabled"
  },
  "id": "/subscriptions/652a3cac-df0f-4b90-a24d-d281ce918841/resourceGroups/rg-practica4/pr
oviders/Microsoft.ContainerRegistry/registries/acrpractica4",
  "identity": null,
  "location": "westeurope",
  "loginServer": "acrpractica4.azurecr.io",
  "metadataSearch": "Disabled",
  "name": "acrpractica4",
  "networkRuleBypassOptions": "AzureServices",
  "networkRuleSet": null,
```

```
PS C:\Users\j0626> az acr login -n acrpractica4
Login Succeeded
```

1.4.4. Etiquetado y subida de la imagen

```
docker tag practica4-api acrpractica4.azurecr.io/practica4-api:v1
docker push acrpractica4.azurecr.io/practica4-api:v1
```

```
PS C:\Users\j0626> docker tag practica4-api acrpractica4.azurecr.io/practica4-api:v1
```

```
PS C:\Users\j0626> docker push acrpractica4.azurecr.io/practica4-api:v1
The push refers to repository [acrpractica4.azurecr.io/practica4-api]
be436228046b: Pushed
ec781dee3f47: Pushed
03046f196102: Pushed
5f4172260b6d: Pushed
3f33cdabfbd0: Pushed
76cca8ae4343: Pushed
b186029d0aef: Pushed
e2a2a5e6ffbf: Pushed
8656d925a408: Pushed
36a355aac46f: Pushed
c1fd66dc2c3f: Pushed
45643b17beb9: Pushed
eecee41f47a6: Pushed
v1: digest: sha256:b3f3ec869441a2c4aeaab5cbd497a00e1888a5ca202fa226b00a2d038b5d0ca3 size: 8
56
```

1.4.5. Creación de entorno y aplicación

```
az containerapp env create -n env-practica4 -g rg-practica4 -l westeurope
az containerapp create -n ca-practica4 -g rg-practica4 --environment env-practica4 --image acrpractica4.azurecr.io/practica4-api:v1 --target-port 8000 --ingress external --registry-server acrpractica4.azurecr.io --cpu 1 --memory 2Gi --min-replicas 1 --max-replicas 3 --secrets api-key=%KhJh-yj44k[RMuJpy --env-vars API_KEY=secretref:api-key RATE_LIMIT=10
```

```
PS C:\Users\j0626> az containerapp env create -n env-practica4 -g rg-practica4 -l westeurop
e
No Log Analytics workspace provided.
Subscription 652a3cac-df0f-4b90-a24d-d281ce918841 is not registered for the Microsoft.Opera
tionalInsights resource provider. Please run "az provider register -n Microsoft.Operational
Insights --wait" to register your subscription.
PS C:\Users\j0626> az provider register -n Microsoft.OperationalInsights --wait
PS C:\Users\j0626> az containerapp env create -n env-practica4 -g rg-practica4 -l westeurop
e
Registering resource provider Microsoft.App ...
No Log Analytics workspace provided.
Generating a Log Analytics workspace with name "workspace-rgpractica4MEEL"

Container Apps environment created. To deploy a container app, use: az containerapp create
--help

{
  "id": "/subscriptions/652a3cac-df0f-4b90-a24d-d281ce918841/resourceGroups/rg-practica4/pr
oviders/Microsoft.App/managedEnvironments/env-practica4",
  "location": "West Europe",
  "name": "env-practica4",
  "properties": {
    "appInsightsConfiguration": null,
    "appLogsConfiguration": {
      "destination": "log-analytics",
      "logAnalyticsConfiguration": {
        "customerId": "11b13c94-9388-45d3-9513-ca4436830af3",
        "sharedKey": null
      }
    },
```

```
PS C:\Users\j0626> az acr update -n acrpractica4 --admin-enabled true
{
  "adminUserEnabled": true,
  "anonymousPullEnabled": false,
  "autoGeneratedDomainNameLabelScope": "Unsecure",
  "creationDate": "2026-03-28T16:44:22.157497+00:00",
  "dataEndpointEnabled": false,
  "dataEndpointHostNames": [],
  "encryption": {
    "keyVaultProperties": null,
    "status": "disabled"
  },
  "id": "/subscriptions/652a3cac-df0f-4b90-a24d-d281ce918841/resourceG
```

```
PS C:\Users\j0626> $acrUser = az acr credential show -n acrpractica4 --query username -o ts
v
```

```
PS C:\Users\j0626> $acrPass = az acr credential show -n acrpractica4 --query "passwords[0].
value" -o tsv
```

```
PS C:\Users\j0626> az containerapp create -n ca-practica4 -g rg-practica4 --environment env
-practica4 --image acrpractica4.azurecr.io/practica4-api:v1 --target-port 8000 --ingress ex
ternal --registry-server acrpractica4.azurecr.io --registry-username $acrUser --registry-pa
ssword $acrPass --cpu 1 --memory 2Gi --min-replicas 1 --max-replicas 3 --secrets "api-key=%
KhJh-yj44k[RMuJpy" --env-vars "API_KEY=secretref:api-key" "RATE_LIMIT=10"
Adding registry password as a secret with name "acrpractica4azurecrio-acrpractica4"

Container app created. Access your app at https://ca-practica4.wittycoast-99cc6956.westeuro
pe.azurecontainerapps.io/

{
  "id": "/subscriptions/652a3cac-df0f-4b90-a24d-d281ce918841/resourceGroups/rg-practica4/pr
oviders/Microsoft.App/containerapps/ca-practica4",
  "identity": {
    "type": "None"
  },
  "location": "West Europe",
  "name": "ca-practica4",
  "properties": {
    "configuration": {
      "activeRevisionsMode": "Single",
```

1.4.6. Obtención de URL publica

```
az containerapp show -n ca-practica4 -g rg-practica4 --query
properties.configuration.ingress.fqdn -o tsv
```

```
PS C:\Users\j0626> az containerapp show -n ca-practica4 -g rg-practica4 --query properties.
configuration.ingress.fqdn -o tsv
ca-practica4.wittycoast-99cc6956.westeurope.azurecontainerapps.io
PS C:\Users\j0626>
```

Con la URL resultante, la API queda disponible públicamente mediante HTTPS.

1.5. Verificacion funcional

1.5.1. Health check

```
curl -X GET https://<URL_PUBLICA>/health
```

Respuesta esperada (ejemplo):

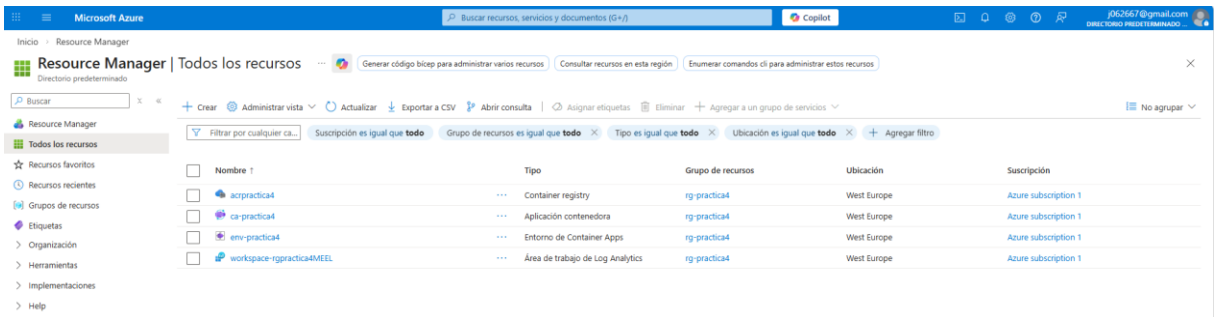
```
{
  "status": "ok",
  "model_loaded": true,
  "tokenizer_loaded": true,
  "rate_limiter_enabled": true
}
```

```
PS C:\Users\j0626> curl.exe -X GET "https://ca-practica4.wittycoast-99cc6956.westeurope.azu
recontainerapps.io/health"
{"status":"ok","model_loaded":true,"tokenizer_loaded":true,"rate_limiter_enabled":true}
```

1.5.2. Generacion de texto

```
curl -X POST https://<URL_PUBLICA>/generate \
  -H "X-API-Key: TU_API_KEY_SEGURA" \
  -H "Content-Type: application/json" \
  -d '{"seed":"la inteligencia
artificial","n_words":20,"strategy":"sampling","temperature":0.8,"top_k":40
}'
```

```
PS C:\Users\j0626> curl -X POST https://ca-practica4.wittycoast-99cc6956.westeurope.azureco
ntainerapps.io/generate -H "X-API-Key: %KhJh-yj44k[RMuJpy" -H "Content-Type: application/js
on" -d '{"seed":"la inteligencia artificial","n_words":20,"strategy":"sampling","temperatur
e":0.8,"top_k":40}'
{"seed":"la inteligencia artificial","generated_text":"la inteligencia artificial (ia) empe
zó a desarrollarse para llenar sus propios primeros días después de que el robot logre cons
truir un mundo de","strategy":"sampling","n_words_generated":20,"elapsed_ms":8493.85}
```



1.6. Medidas de seguridad aplicadas

Se documentan al menos dos medidas para prevenir abusos en la API.

1.6.1. Autenticacion por API Key

- El endpoint de generación exige la cabecera X-API-Key.

- Peticiones sin clave o con clave incorrecta reciben `401 Unauthorized`.

- La clave no se fija dentro del Dockerfile como valor sensible.

1.6.2. Limitación de tasa (Rate Limiting)

- Se controla el número de peticiones por minuto usando RATE_LIMIT.

- Reduce riesgo de abuso, automatizaciones agresivas y saturación del servicio.

1.6.3. Medidas complementarias recomendadas

- Gestión de secretos en la nube (Azure secrets) en lugar de variables hardcodeadas.

- Uso obligatorio de HTTPS en todos los endpoints públicos.

- Restricciones CORS por dominio en escenarios productivos.

- Observabilidad y alertas para detectar picos anormales de tráfico.